

Contents

Data Structures and Algorithms	2
Complete Topic List	2
Chapter Files	2
1.1 Arrays	3
1.2 Stacks	6
1.3 Queues	11
1.4 Linked Lists	15
1.5 Additional Linked List Problems	22
1.6 Tower of Hanoi (Classic Recursive Problem)	26
1.7 Dynamic Array Growth Analysis	29
Cheat-Sheet	31
Review Checklist	31
2.1 Introduction to Trees	32
2.2 Binary Trees	33
2.3 Binary Tree Traversals	34
2.4 Binary Search Trees (BST)	37
2.5 Balanced Trees and AVL Trees	41
2.6 Heaps	44
2.6 B+ Trees	47
2.7 Common Tree Operations Summary	53
2.8 Practice Questions	53
Cheat-Sheet	54
Review Checklist	55
3.1 Introduction to Hashing	55
3.2 Hash Functions	56
3.3 Collisions	58
3.4 Chaining (Separate Chaining)	59
3.5 Open Addressing	62
3.6 Linear Probing	63
3.7 Quadratic Probing	65
3.8 Double Hashing	67
3.9 Comparison of Collision Resolution Methods	68
3.10 HashMap in Java	69
3.11 HashSet in Java	71
3.12 Hashing Applications	71
3.13 Edge Cases & Traps	72
3.14 Practice Questions	74
Cheat-Sheet	75
Review Checklist	76
4.1 Introduction to Graphs	76
4.2 Graph Representations	77
4.3 Breadth-First Search (BFS)	80
4.4 Depth-First Search (DFS)	83
4.5 BFS vs DFS Comparison	86
4.6 Minimum Spanning Tree (MST)	86
4.7 Dijkstra's Shortest Path Algorithm	90

4.7 Bellman-Ford Algorithm	93
4.8 Topological Sort	97
4.9 Graph Cycle Detection	100
4.10 Edge Cases & Traps	101
4.11 Practice Questions	103
Cheat-Sheet	104
Review Checklist	105

Data Structures and Algorithms

Complete Topic List

1. Linear Data Structures

- Arrays (static, dynamic, 2D, multi-dimensional)
- Stacks (LIFO, push, pop, peek, applications)
- Queues (FIFO, enqueue, dequeue, front, rear)
- Linked lists (singly, doubly, circular)

2. Trees and Hierarchical Structures

- Binary trees (traversals: inorder, preorder, postorder, level-order)
- Binary heaps (min-heap, max-heap, heapify, heap sort)
- Priority queues (implementation with heap)
- Binary search trees (BST) (insert, delete, search, traversals)
- AVL trees (balance factor, rotations: LL, RR, LR, RL)
- B+ trees (leaf nodes linked, insert, delete)

3. Hashing Techniques

- Hash tables (key-value pairs, hash function)
- Hash functions (division, multiplication, universal hashing)
- Collision resolution by chaining (linked list chaining)
- Open addressing and probe sequences (linear, quadratic, double hashing)

4. Graphs

- Representations of graphs
 - Adjacency matrix
 - Adjacency list
- Graph traversals (BFS, DFS)
- Shortest path algorithms (Dijkstra, Bellman-Ford)
- Minimum spanning tree (Prim, Kruskal)

Chapter Files

- `chapters/chap1_linear_data_structures.md`
- `chapters/chap2_trees_hierarchical.md`

- chapters/chap3_hashing.md
- chapters/chap4_graphs.md # Chapter 1: Linear Data Structures

1.1 Arrays

Definition / Theoretical Background

An **array** is a contiguous block of memory containing elements of the same type, accessed by integer indices. It is the most fundamental data structure.

Key Properties: - **Fixed size** (static) or **dynamic** (growable) - **Contiguous memory** - all elements stored side-by-side - **O(1) random access** - any element accessed in constant time - **Homogeneous** - all elements are the same type

Memory Layout of Array:

```
Address:  0x1000  0x1004  0x1008  0x100C  0x1010
          +-----+-----+-----+-----+-----+
Index:    |   0   |   1   |   2   |   3   |   4   |
          |  10   |  20   |  30   |  40   |  50   |
          +-----+-----+-----+-----+-----+
          ~
arr[0] = base_address + 0 * sizeof(element)
arr[3] = base_address + 3 * sizeof(element) = 0x100C
```

Static vs Dynamic Arrays

Aspect	Static Array	Dynamic Array
Size	Fixed at compile time	Grows at runtime
Language	C/C++ (int arr[100])	Java (ArrayList), C++ (vector), Python (list)
Memory	Stack (usually)	Heap
Resize	Cannot resize	Can resize (copy to new larger array)
Performance	Faster (no heap allocation)	Slower (may require copying)

Array Operations and Complexity

Operation	Time Complexity	Notes
Access by index	O(1)	Direct calculation: addr = base + i * size
Search (unsorted)	O(n)	Linear scan
Search (sorted)	O(log n)	Binary search
Insert at end	O(1) amortized	Dynamic arrays only
Insert at middle	O(n)	Shift elements
Delete from middle	O(n)	Shift elements
Delete at end	O(1)	Just reduce size

Arrays in Different Languages

```

// Declaration
int arr[5];           // Uninitialized (garbage)
int arr[5] = {0};     // All zeros
int arr[5] = {1, 2, 3}; // Partial init: {1, 2, 3, 0, 0}
int arr[] = {1, 2, 3}; // Compiler infers size = 3

// Access
arr[0] = 10;
printf("%d", arr[2]);

// Multidimensional
int matrix[3][4]; // 3 rows, 4 columns
matrix[0][0] = 1;

```

C/C++ Static Array

```

// Declaration
int[] arr = new int[5];           // Default values (0)
int[] arr = {1, 2, 3, 4, 5};     // Init with values
String[] [] matrix = new String[3][4]; // 2D array

// Arrays have .length property
System.out.println(arr.length); // 5

// Cannot resize - create new array if needed
int[] larger = new int[10];
System.arraycopy(arr, 0, larger, 0, arr.length);

```

Java Arrays

```

# Python list is a dynamic array (resizable)
lst = [1, 2, 3]           # Create with values
lst = []                  # Empty list
lst = list([1, 2, 3])     # From iterable

# Append is amortized O(1)
lst.append(4)              # [1, 2, 3, 4]

# Insert at middle is O(n)
lst.insert(1, 99)          # [1, 99, 2, 3, 4]

# List is homogeneous but can hold mixed types
lst = [1, "hello", 3.14]  # Works but not recommended

```

Python Lists (Dynamic Arrays)

2D Arrays (Matrices)

```
// C: Row-major order (rows stored contiguously)
int mat[3][4] = {           // 3 rows, 4 columns
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Memory layout (row-major):
// [1][2][3][4][5][6][7][8][9][10][11][12]
// 0  1  2  3  4  5  6  7  8  9  10 11 (linear indices)

// Access
mat[1][2]; // Row 1, Column 2 = 7
*(mat[1] + 2); // Same thing (pointer arithmetic)

// Address calculation for mat[i][j]:
// address = base + (i * numCols + j) * sizeof(element)
```

Edge Cases & Traps

Trap 1: Array Index Out of Bounds

```
int arr[5] = {1, 2, 3, 4, 5};
printf("%d", arr[5]); // DANGER! arr[5] doesn't exist!
// C doesn't check bounds - may print garbage or crash!
// Java/Python would throw ArrayIndexOutOfBoundsException
```

[WARNING] **Professor Trap:** Array indices 0 to size-1. Accessing arr[size] is always wrong!

Trap 2: Off-By-One Error in Loops

```
// WRONG: Goes one past end
for (int i = 0; i <= 5; i++) { // <= is wrong!
    printf("%d ", arr[i]); // arr[5] is out of bounds!
}

// CORRECT:
for (int i = 0; i < 5; i++) { // < (not <=)
    printf("%d ", arr[i]);
}
```

Trap 3: Static vs Dynamic Array Size

```
# Python list is dynamic but has overhead
import sys
lst = [1, 2, 3]
sys.getsizeof(lst) # ~64 bytes (overhead for dynamic resizing)
```

```
# vs C static array:
# int arr[3]; // Exactly 12 bytes on 4-byte int system
```

Trap 4: Array Name is NOT a Pointer (C)

```
int arr[5] = {1, 2, 3, 4, 5};

// arr is NOT a pointer! It's the array object.
// But in most contexts, decays to pointer to first element
printf("%zu", sizeof(arr)); // 20 (entire array!)
// vs
int *p = arr;
printf("%zu", sizeof(p)); // 8 (pointer size!)

// Function parameter: arr[] actually becomes pointer!
void func(int arr[]) {
    sizeof(arr); // Gives pointer size, not array size!
}
```

Trap 5: Comparing Arrays with ==

```
int a[5] = {1, 2, 3, 4, 5};
int b[5] = {1, 2, 3, 4, 5};

if (a == b) { // ALWAYS FALSE! Compares addresses, not contents!
    // Never executes!
}

// Must compare element by element:
for (int i = 0; i < 5; i++) {
    if (a[i] != b[i]) { /* not equal */ }
}
```

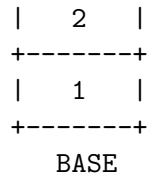
1.2 Stacks

Definition / Theoretical Background

A **stack** is a LIFO (Last-In, First-Out) data structure where the last element added is the first removed.

STACK (LIFO):

```
+-----+
|  5  | <- TOP (last in, first out)
+-----+
|  4  |
+-----+
|  3  |
+-----+
```



Operations:

- push(item): Add to top ($O(1)$)
- pop(): Remove from top ($O(1)$)
- peek(): View top without removing ($O(1)$)
- isEmpty(): Check if empty ($O(1)$)
- isFull(): Check if full (for fixed size) ($O(1)$)

Stack Implementation

```

class StackArray {
    private int[] arr;
    private int top;
    private int capacity;

    StackArray(int size) {
        arr = new int[size];
        capacity = size;
        top = -1;
    }

    void push(int item) {
        if (top == capacity - 1) {
            throw new StackOverflowError();
        }
        arr[++top] = item;
    }

    int pop() {
        if (top == -1) {
            throw new IllegalStateException("Stack empty");
        }
        return arr[top--];
    }

    int peek() {
        if (top == -1) {
            throw new IllegalStateException("Stack empty");
        }
        return arr[top];
    }
}

```

```

    boolean isEmpty() { return top == -1; }
}

```

Java Stack (using Array)

```

class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)  # O(1) amortized

    def pop(self):
        if self.is_empty():
            raise IndexError("Stack empty")
        return self.items.pop()  # O(1)

    def peek(self):
        if self.is_empty():
            raise IndexError("Stack empty")
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

# Or just use Python list directly:
stack = []
stack.append(1)  # push
stack.pop()      # pop

```

Python Stack (using List)

Stack Applications

```

class UndoManager {
    private Stack<String> undoStack = new Stack<>();
    private Stack<String> redoStack = new Stack<>();

    void edit(String change) {
        undoStack.push(change);
        redoStack.clear();  // Clear redo on new edit
    }

    void undo() {
        if (!undoStack.isEmpty()) {
            String change = undoStack.pop();

```

```

        redoStack.push(change);
    }
}

```

1. Undo/Redo Operations

```

boolean isBalanced(String s) {
    Stack<Character> stack = new Stack<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}

// Test:
isBalanced("({[]})"); // true
isBalanced("([]]"); // false

```

2. Balanced Parentheses / Brackets

```

// Recursion uses call stack
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

/*
factorial(4) call stack:
factorial(4) -> pushes frame
factorial(3) -> pushes frame
factorial(2) -> pushes frame
factorial(1) -> returns 1, pops frame
returns 2 to factorial(2)
returns 6 to factorial(3)

```

```
    returns 24 to factorial(4)  
*/
```

3. Function Call Stack (Recursion)

```
// Infix to Postfix conversion  
String infixToPostfix(String infix) {  
    Stack<Character> stack = new Stack<>();  
    String result = "";  
  
    for (char c : infix.toCharArray()) {  
        if (Character.isLetterOrDigit(c)) {  
            result += c;  
        } else if (c == '(') {  
            stack.push(c);  
        } else if (c == ')') {  
            while (!stack.isEmpty() && stack.peek() != '(') {  
                result += stack.pop();  
            }  
            stack.pop(); // Remove '('  
        } else { // Operator  
            while (!stack.isEmpty() && precedence(c) <= precedence(stack.peek())) {  
                result += stack.pop();  
            }  
            stack.push(c);  
        }  
    }  
  
    while (!stack.isEmpty()) {  
        result += stack.pop();  
    }  
    return result;  
}
```

4. Expression Evaluation (Postfix/Infix to Postfix)

Edge Cases & Traps

Trap 1: Stack Overflow (Unlimited Recursion)

```
// Recursive without base case - STACK OVERFLOW!  
int badRecursion(int n) {  
    return badRecursion(n + 1); // Infinite recursion!  
    // Each call pushes to stack until overflow!  
}
```

[WARNING] **Professor Trap:** Every recursive function needs a base case!

Trap 2: Stack Underflow

```
stack.pop(); // If stack is empty, throws exception!
```

[WARNING] **Professor Trap:** Always check `isEmpty()` before `pop()`!

Trap 3: Forgetting to Pop All Elements

```
# Memory leak in stack implementation
def push(item):
    stack.append(item) # Forgot to implement!

def pop():
    if len(stack) > 0:
        return stack.pop() # Should also check empty!
```

1.3 Queues

Definition / Theoretical Background

A **queue** is a FIFO (First-In, First-Out) data structure where the first element added is the first removed.

QUEUE (FIFO):

```

FRONT/HEAD                                     REAR/TAIL
+-----+-----+-----+-----+-----+
|  1   |  2   |  3   |  4   |  5   |
+-----+-----+-----+-----+
      ^                                     ^
first in                               last in
first out                             last out

```

Operations:

- enqueue(item): Add to rear ($O(1)$)
- dequeue(): Remove from front ($O(1)$)
- peek(): View front ($O(1)$)
- isEmpty(): Check if empty ($O(1)$)

Queue Implementations

```
class QueueArray {
    private int[] arr;
    private int front, rear, size;

    QueueArray(int capacity) {
        arr = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }
}
```

```

}

void enqueue(int item) {
    if (size == arr.length) {
        throw new RuntimeException("Queue full");
    }
    arr[++rear] = item;
    size++;
}

int dequeue() {
    if (size == 0) {
        throw new RuntimeException("Queue empty");
    }
    int item = arr[front++];
    size--;
    return item;
}

int peek() {
    if (size == 0) {
        throw new RuntimeException("Queue empty");
    }
    return arr[front];
}
}

```

Simple Array Queue (with issues)

```

class CircularQueue {
    private int[] arr;
    private int front, rear, size;
    private int capacity;

    CircularQueue(int capacity) {
        arr = new int[capacity];
        this.capacity = capacity;
        front = 0;
        rear = -1;
        size = 0;
    }

    void enqueue(int item) {
        if (size == capacity) {
            throw new RuntimeException("Queue full");
        }
    }
}

```

```

        rear = (rear + 1) % capacity;
        arr[rear] = item;
        size++;
    }

    int dequeue() {
        if (size == 0) {
            throw new RuntimeException("Queue empty");
        }
        int item = arr[front];
        front = (front + 1) % capacity;
        size--;
        return item;
    }
}

```

Circular Queue (Efficient)

Queue Types

Type	Description	Use Case
Simple Queue (FIFO)	Standard first-in-first-out	Task scheduling
Circular Queue	Wraps around end to beginning	Buffer, BFS
Priority Queue	Highest priority dequeued first	Dijkstra, Huffman
Deque (Double-ended)	Add/remove from both ends	Palindrome, undo

Queue Applications

```

void bfs(int start, List<Integer>[] graph) {
    Queue<Integer> queue = new LinkedList<>();
    boolean[] visited = new boolean[graph.length];

    queue.offer(start);
    visited[start] = true;

    while (!queue.isEmpty()) {
        int vertex = queue.poll();
        System.out.print(vertex + " ");

        for (int neighbor : graph[vertex]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}

```

```

    }
}

```

1. BFS (Breadth-First Search)

```

class TaskScheduler {
    private Queue<Task> taskQueue = new LinkedList<>();

    void addTask(Task task) {
        taskQueue.offer(task);
    }

    void executeNext() {
        if (!taskQueue.isEmpty()) {
            Task task = taskQueue.poll();
            task.execute();
        }
    }
}

```

2. CPU Task Scheduling

Deque (Double-Ended Queue)

```

from collections import deque

d = deque()

# Add to both ends - O(1)
d.append(1)      # right
d.appendleft(0) # left
d.append(2)      # right

# Remove from both ends - O(1)
d.pop()          # rightmost
d.popleft()      # leftmost

# deque can be used as stack or queue
# Also for sliding window problems

```

Edge Cases & Traps

Trap 1: Array Queue Index Issues

```

// Without circular wrapping, array fills up!
// enqueue(1), enqueue(2), ..., enqueue(n)
// After n enqueues, queue is "full"

```

```
// Even if you dequeue(), queue is still "full"!
```

```
// Solution: Use circular queue!
```

[WARNING] **Professor Trap:** Simple array queue wastes space after dequeue!

Trap 2: Empty Queue Dequeue

```
queue.dequeue(); // If empty, throws exception!
```

[WARNING] **Professor Trap:** Always check isEmpty() before dequeue!

Trap 3: Queue vs Stack Selection

```
// When to use Queue:  
// - Print tasks in order received  
// - BFS traversal  
// - Task scheduling
```

```
// When to use Stack:  
// - Undo operations  
// - DFS traversal  
// - Expression evaluation
```

[WARNING] **Professor Trap:** Don't confuse queue (FIFO) with stack (LIFO)!

1.4 Linked Lists

Definition / Theoretical Background

A **linked list** is a linear data structure where elements (nodes) are connected by pointers/references. Each node contains data and a pointer to the next node (singly) or both next and previous (doubly).

SINGLY LINKED LIST:

HEAD	TAIL
->	->
+-----+-----+	+-----+-----+
1 [*]-+----->	2 [*]-+-----> 3 NULL
+-----+-----+	+-----+-----+
data next	data next

DOUBLY LINKED LIST:

NULL
^
+-----+-----+
<--- 1 [*]-+---> 2 [*]-+---> 3 [*]--> NULL
+-----+-----+
prev

Linked List vs Array

Aspect	Linked List	Array
Memory	Scattered (nodes connected by pointers)	Contiguous block
Access	$O(n)$ - must traverse	$O(1)$ - direct access
Insert at head	$O(1)$	$O(n)$ - must shift
Insert at tail	$O(1)$ with tail pointer	$O(1)$ amortized
Delete from middle	$O(1)$ if at known position	$O(n)$
Memory overhead	Higher (next/prev pointers)	Lower
Cache locality	Poor	Excellent

Singly Linked List Implementation (Java)

```
class Node {
    int data;
    Node next;

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}

class LinkedList {
    private Node head;
    private Node tail;
    private int size;

    // Insert at head - O(1)
    void insertAtHead(int data) {
        Node newNode = new Node(data);
        newNode.next = head;
        head = newNode;
        if (tail == null) tail = head;
        size++;
    }

    // Insert at tail - O(1) with tail pointer
    void insertAtTail(int data) {
        Node newNode = new Node(data);
        if (tail == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            tail = newNode;
        }
    }
}
```

```

        size++;
    }

    // Delete first occurrence - O(n)
    void delete(int data) {
        if (head == null) return;

        if (head.data == data) {
            head = head.next;
            if (head == null) tail = null;
            size--;
            return;
        }

        Node current = head;
        while (current.next != null) {
            if (current.next.data == data) {
                current.next = current.next.next;
                if (current.next == null) tail = current;
                size--;
                return;
            }
            current = current.next;
        }
    }

    // Search - O(n)
    boolean search(int data) {
        Node current = head;
        while (current != null) {
            if (current.data == data) return true;
            current = current.next;
        }
        return false;
    }

    // Reverse - O(n)
    void reverse() {
        Node prev = null;
        Node current = head;
        tail = head;

        while (current != null) {
            Node next = current.next;
            current.next = prev;
            prev = current;
            current = next;
        }
    }

```

```

    }
    head = prev;
}
}

```

Doubly Linked List (Java)

```

class DNode {
    int data;
    DNode prev;
    DNode next;

    DNode(int data) {
        this.data = data;
    }
}

class DoublyLinkedList {
    private DNode head, tail;
    private int size;

    void insertAtHead(int data) {
        DNode newNode = new DNode(data);
        if (head == null) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
        size++;
    }

    void insertAtTail(int data) {
        DNode newNode = new DNode(data);
        if (tail == null) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
        size++;
    }

    void delete(DNode node) {
        if (node.prev != null) node.prev.next = node.next;
    }
}

```

```

        else head = node.next;

        if (node.next != null) node.next.prev = node.prev;
        else tail = node.prev;

        size--;
    }
}

```

Circular Linked List

```

class CircularList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if not self.head:
            new_node.next = new_node
            self.head = new_node
        else:
            # Find last node and insert after it
            last = self.head
            while last.next != self.head:
                last = last.next
            last.next = new_node
            new_node.next = self.head

```

Linked List Applications

```

class BrowserHistory {
    private LinkedList<String> history;
    private int current;

    BrowserHistory(String homepage) {
        history = new LinkedList<>();
        history.add(homepage);
        current = 0;
    }

    void visit(String url) {
        // Clear forward history
        while (history.size() - 1 > current) {
            history.removeLast();
        }
        history.add(url);
    }
}

```

```

        current++;
    }

    String back() {
        if (current > 0) {
            current--;
        }
        return history.get(current);
    }

    String forward() {
        if (current < history.size() - 1) {
            current++;
        }
        return history.get(current);
    }
}

```

1. Browser History (Back/Forward)

```

class Playlist {
    private DNode current;

    void addSong(String song) {
        DNode newNode = new DNode(song);
        if (current == null) {
            current = newNode;
        } else {
            newNode.prev = current;
            newNode.next = current.next;
            if (current.next != null) current.next.prev = newNode;
            current.next = newNode;
            current = newNode;
        }
    }

    String next() {
        if (current.next != null) current = current.next;
        return current.data;
    }

    String prev() {
        if (current.prev != null) current = current.prev;
        return current.data;
    }
}

```

2. Music Playlist (Next/Previous)

Edge Cases & Traps

Trap 1: Losing Head Pointer

```
// WRONG: Head is lost!
void badInsert(Node **head, int data) {
    Node *newNode = malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = *head;
    *head = newNode; // Must update *head!
}

// If you forget: *head = newNode, head is not updated in caller!
```

Trap 2: Null Pointer in Loop

```
// WRONG: Doesn't handle null correctly
while (current.next != null) {
    current = current.next; // Works until last node
}

// If list is empty, current = null, then current.next CRASH!
```

[WARNING] **Professor Trap:** Always check if list is empty before traversing!

Trap 3: Memory Leak in C (Not Freeing)

```
// After deleting all nodes
while (head != NULL) {
    Node *temp = head;
    head = head->next;
    // free(temp); // FORGOT! Memory leak!
}

// Fix: Always free!
while (head != NULL) {
    Node *temp = head;
    head = head->next;
    free(temp);
}
```

Trap 4: Dangling Pointers

```
Node *head = malloc(sizeof(Node));
Node *second = malloc(sizeof(Node));
head->next = second;
second->next = NULL;
free(second); // second is freed
// head->next is now DANGLING (points to freed memory!)
head->next = NULL; // Fix: set to NULL before/after free
```

Trap 5: Wrong Tail Update

```
// If you delete the tail but forget to update it:
if (current.next == tail) {
    tail = current;
    tail.next = null;
}
```

[WARNING] **Professor Trap:** When deleting last node, must update tail pointer!

[WARNING] **Professor Trap:** When deleting last node, must update tail pointer!

1.5 Additional Linked List Problems

Finding the Middle Node (Tortoise and Hare Algorithm)

```
// O(n) time, O(1) space - only one pass!
// Uses two pointers: slow moves 1 step, fast moves 2 steps
// When fast reaches end, slow is at middle

Node findMiddle(Node head) {
    if (head == null) return null;

    Node slow = head;
    Node fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow; // Middle node!
}

// Variations:
// - For even length list, returns SECOND middle
// - For even length, to return FIRST middle: use fast.next.next != null
```

Detecting Cycle in Linked List (Floyd's Algorithm)

```
// Floyd's Cycle Detection Algorithm
// Also called "tortoise and hare" (slow and fast pointers)

boolean hasCycle(Node head) {
    if (head == null) return false;

    Node slow = head; // Moves 1 step
```

```

Node fast = head;  // Moves 2 steps

while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;

    if (slow == fast) {
        return true;  // Cycle detected!
    }
}

return false;  // No cycle
}

// Step-by-step trace:
// List: 1 -> 2 -> 3 -> 4 -> 5 -> 3 (cycle back to 3)
//
// Step 1: slow=1, fast=1
// Step 2: slow=2, fast=3
// Step 3: slow=3, fast=5
// Step 4: slow=4, fast=3 (from 5.next = 3)
// Step 5: slow=5, fast=5 (from 3.next.next = 5)
// slow == fast -> cycle detected!

```

Finding Cycle Start (Floyd's Algorithm Extended)

```

// After detecting cycle, find where it starts

Node findCycleStart(Node head) {
    if (head == null) return null;

    Node slow = head;
    Node fast = head;

    // First: detect cycle
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) break;
    }

    if (fast == null || fast.next == null) return null;  // No cycle

    // Second: find cycle start
    // Reset slow to head, keep fast at meeting point
    // Move both one step at a time, they'll meet at cycle start
    slow = head;

```

```

while (slow != fast) {
    slow = slow.next;
    fast = fast.next;
}

return slow; // Cycle start node
}

// Mathematical proof:
// Let d = distance from head to cycle start
// Let c = length of cycle
// When slow and fast meet:
// - slow traveled: d + x
// - fast traveled: d + x + n*c (for some n)
// But fast traveled twice as much: 2(d + x) = d + x + n*c
// Therefore: d + x = n*c
// So d = n*c - x = (n-1)*c + (c-x)
// When we reset slow to head and move both 1 step:
// After d steps, slow reaches cycle start
// Fast is at: current + d = (d+x) + d = 2d + x = 2d + (n*c - d) = d + n*c
// Which is same as d (mod c) - the cycle start!

```

LRU Cache Implementation (HashMap + Doubly Linked List)

```

class LRUCache {
    private int capacity;
    private Map<Integer, Node> cache; // Key -> Node
    private Node head; // Dummy head (LRU is right after head)
    private Node tail; // Dummy tail (MRU is right before tail)

    LRUCache(int capacity) {
        this.capacity = capacity;
        this.cache = new HashMap<>();
        head = new Node(0, 0);
        tail = new Node(0, 0);
        head.next = tail;
        tail.prev = head;
    }

    // Get: O(1)
    int get(int key) {
        Node node = cache.get(key);
        if (node == null) return -1;

        // Move to MRU position (right before tail)
        remove(node);
        addToEnd(node);
    }
}

```

```

        return node.value;
    }

    // Put: O(1)
    void put(int key, int value) {
        if (cache.containsKey(key)) {
            Node node = cache.get(key);
            node.value = value;
            remove(node);
            addToEnd(node);
        } else {
            if (cache.size() == capacity) {
                // Remove LRU (right after head)
                Node lru = head.next;
                remove(lru);
                cache.remove(lru.key);
            }
            Node newNode = new Node(key, value);
            cache.put(key, newNode);
            addToEnd(newNode);
        }
    }

    void remove(Node node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    void addToEnd(Node node) {
        node.prev = tail.prev;
        node.next = tail;
        tail.prev.next = node;
        tail.prev = node;
    }
}

/*
LRU Cache Visualization:

HEAD <-> [LRU] <-> [ ... ] <-> [MRU] <-> TAIL

Get(5): Moves node 5 to MRU position
        HEAD <-> [LRU] <-> [ ... ] <-> [5] <-> TAIL

Put(6, val): If full, removes LRU first
              Then adds new node at MRU position

```

Why HashMap + Doubly Linked List?

- *HashMap: $O(1)$ lookup by key*
 - *Doubly Linked List: $O(1)$ insertion/deletion (with node reference)*
 - *Combined: $O(1)$ get and put operations!*
- */*

1.6 Tower of Hanoi (Classic Recursive Problem)

Problem Statement

TOWER OF HANOI:

Three pegs: Source, Auxiliary, Destination

Goal: Move n disks from Source to Destination

Rules:

1. Only one disk can be moved at a time
2. A larger disk cannot be placed on top of a smaller disk
3. Only the top disk of any peg can be moved

For $n=3$ disks:

[]		
[]		
[]		
=====		
Source	Auxiliary	Destination

Solution (Recursive)

```
void towerOfHanoi(int n, char source, char destination, char auxiliary) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }

    // Move n-1 disks from source to auxiliary (using destination as helper)
    towerOfHanoi(n - 1, source, auxiliary, destination);

    // Move the largest disk from source to destination
    printf("Move disk %d from %c to %c\n", n, source, destination);

    // Move n-1 disks from auxiliary to destination (using source as helper)
    towerOfHanoi(n - 1, auxiliary, destination, source);
}
```

```
// Example for n=3:
// towerOfHanoi(3, 'A', 'C', 'B')
//
// Move disk 1 from A to C
// Move disk 2 from A to B
// Move disk 1 from C to B
// Move disk 3 from A to C
// Move disk 1 from B to A
// Move disk 2 from B to C
// Move disk 1 from A to C
```

Step-by-Step Trace for n=3

Step 1: Move disk 1 from A to C

[]		[]
[]		[]
[]		

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 2: Move disk 2 from A to B

	[]	
[]	[]	[]
[]		

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 3: Move disk 1 from C to B

	[]	
[]	[]	
[]		

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 4: Move disk 3 from A to C

	[]	
[]	[]	[]

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 5: Move disk 1 from B to A

[]		
-----	--	--

	[]	[]

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 6: Move disk 2 from B to C

		[]
[]		[]
		[]

=====

Source	Auxiliary	Destination
--------	-----------	-------------

Step 7: Move disk 1 from A to C

		[]
	[]	[]
		[]

=====

Source	Auxiliary	Destination
--------	-----------	-------------

TOTAL MOVES: $2^3 - 1 = 7$ moves

Mathematical Analysis

Number of moves for n disks: $T(n) = 2^n - 1$

Proof by induction:

- Base case: $T(1) = 1 = 2^1 - 1$
 - Inductive step: Assume $T(k) = 2^k - 1$ for k disks
- For k+1 disks:
- Move k disks from source to auxiliary: $T(k) = 2^k - 1$ moves
 - Move largest disk from source to destination: 1 move
 - Move k disks from auxiliary to destination: $T(k) = 2^k - 1$ moves
- Total: $2(2^k - 1) + 1 = 2^{(k+1)} - 2 + 1 = 2^{(k+1)} - 1$

Time complexity: $O(2^n)$ - exponential!

Space complexity: $O(n)$ - recursion depth

Iterative Solution (Using Stack)

```
class TowerOfHanoiIterative {
    static class Move {
        int disk;
        char from, to;

        Move(int disk, char from, char to) {
```

```

        this.disk = disk;
        this.from = from;
        this.to = to;
    }
}

static void towerOfHanoiIterative(int n) {
    Stack<Move> stack = new Stack<>();
    stack.push(new Move(n, 'A', 'C')); // A=Source, C=Dest, B=Aux

    while (!stack.isEmpty()) {
        Move move = stack.pop();

        if (move.disk == 1) {
            System.out.println("Move disk 1 from " + move.from + " to " + move.to);
        } else {
            // Calculate auxiliary peg
            char aux = getAuxiliary(move.from, move.to);

            // Push in correct order (smaller disks first)
            stack.push(new Move(move.disk - 1, aux, move.to));
            stack.push(new Move(1, move.from, move.to));
            stack.push(new Move(move.disk - 1, move.from, aux));
        }
    }
}

static char getAuxiliary(char from, char to) {
    // A, B, C - find the one that's neither from nor to
    if (from == 'A' && to == 'B' || from == 'B' && to == 'A') return 'C';
    if (from == 'A' && to == 'C' || from == 'C' && to == 'A') return 'B';
    return 'A';
}
}

```

1.7 Dynamic Array Growth Analysis

Why Amortized $O(1)$ for Append

```

// Python list, Java ArrayList, C++ vector all use "doubling" strategy
// When array is full, create new array 2x size and copy all elements

void append(int element) {
    if (size == capacity) {
        capacity = capacity * 2; // Double capacity
        int[] newArr = new int[capacity]; // Allocate new array
    }
}

```

```

        System.arraycopy(arr, 0, newArr, 0, size); // Copy elements
        arr = newArr;
    }
    arr[size++] = element;
}

// Analysis:
// Let n = number of appends
// Cost of copy at size s = s
//
// Sizes: 1, 2, 4, 8, 16, 32, ..., n
// Copies: 0, 1, 2, 4, 8, 16, ..., n/2
//
// Total copies = 1 + 2 + 4 + 8 + ... + n/2 = n - 1
// Total operations = n appends + n - 1 copies = 2n - 1
// Amortized per append = (2n - 1) / n = 2 - 1/n = O(1)

```

Why Doubling (Not Incrementing)?

```

// If we increased by fixed amount (e.g., +1):
// Sizes: 1, 2, 3, 4, 5, ..., n
// Copies: 0, 1, 2, 3, 4, ..., n-1
// Total copies = n(n-1)/2 = O(n^2) -- BAD!

// If we double:
// Total copies = n - 1 = O(n)
// Much better!

// Geometric series: 1 + 2 + 4 + ... + n = 2n - 1

```

Space-Time Tradeoff

```

// Doubling uses more space but fewer copies
// Incrementing uses less space but more copies

// Example: 1 million appends
// Doubling: capacity goes 1, 2, 4, 8, ..., 2^20 = 1,048,576
//           Total extra space = 2n (peak capacity = 2n)
//           Total copies = n = 1,000,000

// Incrementing by 1000: capacity 1000, 2000, 3000, ..., 1,000,000
//           Peak capacity = n + 1000
//           Total copies = sum 1000 + 2000 + ... + 1,000,000 = ~500 million

```

Practice Questions

MCQ 1: What is the time complexity of accessing element at index i in an array? - A) $O(n)$ - B) $O(1)$ - C) $O(\log n)$ - D) $O(n^2)$

MCQ 2: What data structure uses LIFO? - A) Queue - B) Stack - C) Linked List - D) Array

MCQ 3: What is the time complexity of inserting at the head of a singly linked list? - A) $O(n)$ - B) $O(1)$ - C) $O(\log n)$ - D) $O(n^2)$

Short Answer: Explain why arrays have $O(1)$ random access while linked lists have $O(n)$.

Board Coding: Implement a stack using an array. Include push, pop, peek, and isEmpty operations.

Board Coding: Write a function to reverse a singly linked list.

Oral Explanation: Compare arrays and linked lists. When would you choose one over the other?

Solutions

MCQ 1: B) $O(1)$ - Direct address calculation

MCQ 2: B) Stack - Last In First Out

MCQ 3: B) $O(1)$ - Just update head pointer

Short Answer: Arrays store elements in contiguous memory, so address of element $i = \text{base} + i * \text{element_size}$, a single calculation. Linked lists scatter nodes in memory, so to reach node i you must start at head and follow i pointers sequentially.

Cheat-Sheet

Data Structure	Access	Search	Insert (Head)	Insert (Tail)	Delete
Array	$O(1)$	$O(n)$	$O(n)$	$O(1)^*$	$O(n)$
Stack	Top only	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Queue	Front only	$O(n)$	$O(1)$	$O(1)$	$O(1)$
Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)^*$	$O(1)^*$

*With tail pointer

Review Checklist

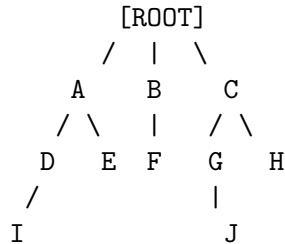
- ☐ **Arrays:** Can you explain $O(1)$ access? Do you know bounds checking?
- ☐ **Stacks:** Can you implement push/pop/peek? Do you know LIFO applications?
- ☐ **Queues:** Can you implement enqueue/dequeue? Do you know FIFO applications?
- ☐ **Linked Lists:** Can you implement insert/delete? Do you know singly vs doubly vs circular?
- ☐ **Time Complexities:** Can you compare all data structures?
- ☐ **Language differences:** Do you know array/list implementations in different languages?
- ☐ **Traps:** Are you aware of off-by-one, null pointer, and memory leak traps? # Chapter 2: Trees and Hierarchical Structures

2.1 Introduction to Trees

Definition / Theoretical Background

A **tree** is a hierarchical data structure consisting of nodes connected by edges, with exactly one path between any two nodes. Trees are non-linear unlike arrays, stacks, and queues which are linear.

TREE STRUCTURE:



Key Terms:

- **ROOT**: Topmost node (has no parent) - node A above
- **CHILD**: Node with a parent - nodes D,E,F,G,H are children of A,B,C
- **PARENT**: Node with one or more children - A is parent of D,E
- **LEAF**: Node with no children - nodes D,I,F,G,J are leaves
- **EDGE**: Connection between two nodes
- **SIBLING**: Nodes with same parent - D and E are siblings
- **DEPTH**: Number of edges from root to node (root = 0)
 - Depth of I = 2 (A->D->I)
- **HEIGHT**: Longest path from node to leaf
 - Height of A = 3 (A->C->H->J)
- **SUBTREE**: Tree formed by a node and its descendants

Why Trees?

- **Hierarchical data**: File systems, XML/HTML DOM, organization charts
- **Efficient searching**: Binary Search Trees give $O(\log n)$ search
- **Auto-sorted data**: BST maintains sorted order
- **Network routing**: Trees model network topologies

Tree vs Other Data Structures

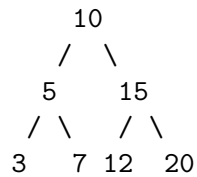
Aspect	Array	Linked List	Tree
Structure	Linear	Linear	Hierarchical
Access	$O(1)$	$O(n)$	$O(n)$ worst
Search	$O(n)$ / $O(\log n)$	$O(n)$	$O(\log n)$ avg
Insert	$O(n)$	$O(1)$	$O(\log n)$ avg
Order	No inherent	Insertion order	Sorted (BST)

2.2 Binary Trees

Definition

A **binary tree** is a tree where each node has at most two children: left child and right child.

BINARY TREE:



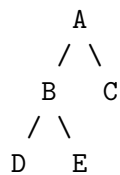
Properties:

- Max nodes at level $i = 2^i$
- Max nodes in tree of height $h = 2^{(h+1)} - 1$
- Min height (with n nodes) = $\text{floor}(\log_2(n))$

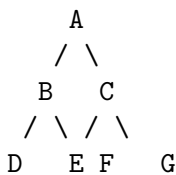
Types of Binary Trees

Type	Definition
Full/Strict	Every node has 0 or 2 children (no 1-child nodes)
Perfect	All internal nodes have 2 children AND all leaves at same level
Complete	All levels filled except possibly last, filled left to right
Balanced	Left and right subtrees height differ by at most 1
Degenerate	Each parent has only one child (like linked list)

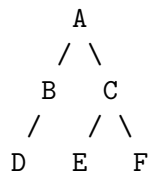
FULL BINARY TREE:



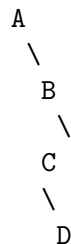
PERFECT BINARY TREE:



COMPLETE BINARY TREE:



DEGENERATE TREE:



Binary Tree Implementation

```
class TreeNode {
    int data;
    TreeNode left;
```

```

TreeNode right;

TreeNode(int data) {
    this.data = data;
    left = null;
    right = null;
}
}

```

Node Structure

```

// For complete binary tree, can use array:
// Parent at i: children at 2i+1 (left), 2i+2 (right)
// Child at i: parent at (i-1)/2

class BinaryTreeArray {
    int[] arr;
    int size;
    int capacity;

    BinaryTreeArray(int capacity) {
        this.capacity = capacity;
        arr = new int[capacity];
        size = 0;
    }

    int parent(int i) { return (i - 1) / 2; }
    int left(int i) { return 2 * i + 1; }
    int right(int i) { return 2 * i + 2; }

    boolean isLeaf(int i) { return left(i) >= size; }
}

```

Array Representation (Complete Binary Tree)

2.3 Binary Tree Traversals

The Four Main Traversals

// Tree for examples:

```

    10
   /  \
  5    15
 /  \  /  \
3   7 12  20

```

Traversal	Order	Result
Inorder (LNR)	Left, Node, Right	3, 5, 7, 10, 12, 15, 20
Preorder (NLR)	Node, Left, Right	10, 5, 3, 7, 15, 12, 20
Postorder (LRN)	Left, Right, Node	3, 7, 5, 12, 20, 15, 10
Level Order (BFS)	Level by level	10, 5, 15, 3, 7, 12, 20

Traversal Implementations

```
class BinaryTree {
    TreeNode root;

    // Inorder: Left -> Root -> Right
    void inorder(TreeNode node) {
        if (node == null) return;
        inorder(node.left);
        System.out.print(node.data + " ");
        inorder(node.right);
    }

    // Preorder: Root -> Left -> Right
    void preorder(TreeNode node) {
        if (node == null) return;
        System.out.print(node.data + " ");
        preorder(node.left);
        preorder(node.right);
    }

    // Postorder: Left -> Right -> Root
    void postorder(TreeNode node) {
        if (node == null) return;
        postorder(node.left);
        postorder(node.right);
        System.out.print(node.data + " ");
    }

    // Level Order (BFS) - uses queue
    void levelOrder() {
        if (root == null) return;

        Queue<TreeNode> queue = new LinkedList<>();
        queue.offer(root);

        while (!queue.isEmpty()) {
            TreeNode node = queue.poll();
            System.out.print(node.data + " ");
        }
    }
}
```

```

        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}

```

When to Use Each Traversal

Traversal	Use Case
Inorder	BST sorted output, getting elements in order
Preorder	Copy tree, prefix expression, serializing tree
Postorder	Delete tree, postfix expression, calculating directory sizes
Level Order	Shortest path, level-by-level processing, BFS

Edge Cases & Traps

Trap 1: Null Pointer in Traversal

```

// WRONG: Will crash if node is null
void badInorder(TreeNode node) {
    inorder(node.left);    // NullPointerException if left is null!
    System.out.println(node.data);
    inorder(node.right);
}

// CORRECT: Always check for null first!
void correctInorder(TreeNode node) {
    if (node == null) return; // Base case!
    correctInorder(node.left);
    System.out.println(node.data);
    correctInorder(node.right);
}

```

[WARNING] **Professor Trap:** Every recursive traversal MUST check for null at start!

Trap 2: Stack Overflow with Deep Trees

```

// For tree with 100,000 nodes deep (degenerate):
// Recursion depth = 100,000 -> STACK OVERFLOW!
void traverse(TreeNode node) {
    traverse(node.left); // May overflow!
}

```

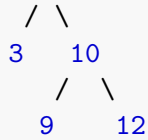
[WARNING] **Professor Trap:** For very deep trees, use iterative approach or increase stack size!

Trap 3: Using Wrong Traversal for BST

```

// BST:
8

```



// If you want sorted output: MUST use INORDER!
// Preorder gives: 8, 3, 10, 9, 12 (not sorted!)
// Inorder gives: 3, 8, 9, 10, 12 (sorted!)

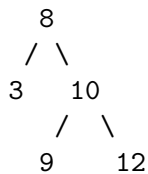
[WARNING] **Professor Trap:** Only inorder traversal gives sorted output for BST!

2.4 Binary Search Trees (BST)

Definition / Theoretical Background

A **Binary Search Tree** is a binary tree with the property: - Left subtree contains nodes less than current node - Right subtree contains nodes greater than current node - Both subtrees must be BSTs themselves

BST:



Valid BST property:

- All nodes in left subtree of 8 are < 8 (just 3) [OK]
- All nodes in right subtree of 8 are > 8 (10, 9, 12) [OK]
- Same rules apply recursively to all subtrees

BST Operations

```

class BST {
    TreeNode root;

    // Search - O(h) where h is height
    TreeNode search(int key) {
        return searchRec(root, key);
    }

    TreeNode searchRec(TreeNode node, int key) {
        if (node == null || node.data == key) return node;

        if (key < node.data) return searchRec(node.left, key);
        else return searchRec(node.right, key);
    }
}

```

```

// Insert - O(h)
void insert(int key) {
    root = insertRec(root, key);
}

TreeNode insertRec(TreeNode node, int key) {
    if (node == null) return new TreeNode(key);

    if (key < node.data) node.left = insertRec(node.left, key);
    else if (key > node.data) node.right = insertRec(node.right, key);
    // If equal, do nothing (no duplicates)

    return node;
}

// Find minimum (leftmost node)
TreeNode minNode(TreeNode node) {
    TreeNode current = node;
    while (current.left != null) current = current.left;
    return current;
}

// Find maximum (rightmost node)
TreeNode maxNode(TreeNode node) {
    TreeNode current = node;
    while (current.right != null) current = current.right;
    return current;
}

// Delete - O(h)
TreeNode delete(TreeNode node, int key) {
    if (node == null) return null;

    if (key < node.data) {
        node.left = delete(node.left, key);
    } else if (key > node.data) {
        node.right = delete(node.right, key);
    } else {
        // Node to delete found

        // Case 1: Leaf node
        if (node.left == null && node.right == null) {
            return null;
        }

        // Case 2: One child
        if (node.left == null) return node.right;
    }
}

```

```

        if (node.right == null) return node.left;

        // Case 3: Two children
        // Find inorder successor (smallest in right subtree)
        TreeNode successor = minNode(node.right);
        node.data = successor.data;
        node.right = delete(node.right, successor.data);
    }
    return node;
}
}

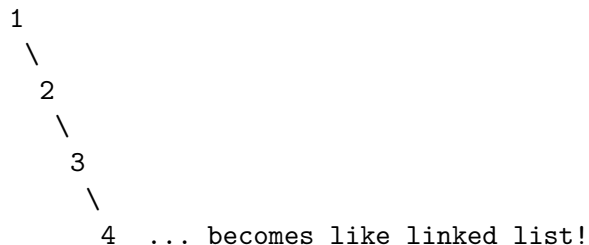
```

BST Time Complexity

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$
Find Min	$O(\log n)$	$O(n)$
Find Max	$O(\log n)$	$O(n)$

Worst case $O(n)$ occurs when tree becomes degenerate (like a linked list):

Degenerate BST (worst case):



BST Applications

```

// Dictionary / Symbol Table
class SymbolTable {
    BST tree = new BST();

    void add(String key, String value) {
        tree.insert(key);
    }

    boolean contains(String key) {
        return tree.search(key) != null;
    }
}

```

```
// Auto-complete
class AutoComplete {
    BST names = new BST();

    List<String> suggestions(String prefix) {
        List<String> result = new ArrayList<>();
        inorderCollect(root, prefix, result);
        return result;
    }
}
```

Edge Cases & Traps

Trap 1: Duplicate Values

```
// BST cannot have duplicates by default!
// If you insert 5 twice:
// First insert: tree has 5
// Second insert: does nothing (or can handle differently)

void insert(int key) {
    root = insertRec(root, key);
}

TreeNode insertRec(TreeNode node, int key) {
    if (node == null) return new TreeNode(key);
    if (key < node.data) node.left = insertRec(node.left, key);
    else if (key > node.data) node.right = insertRec(node.right, key);
    // Equal: do nothing!
    return node;
}
```

[WARNING] **Professor Trap:** BST typically doesn't allow duplicates! Check if your implementation handles them.

Trap 2: Not Handling All Cases in Delete

```
// WRONG: Forgetting case where both children exist
TreeNode delete(TreeNode node, int key) {
    if (node == null) return null;
    if (key < node.data) node.left = delete(node.left, key);
    else if (key > node.data) node.right = delete(node.right, key);
    else {
        // Missing cases for two children!
        if (node.left == null) return node.right;
        if (node.right == null) return node.left;
        // What if both are null? What if both exist?
    }
    return node;
}
```

```
}
```

[WARNING] **Professor Trap:** Delete has THREE cases - leaf, one child, two children!

Trap 3: BST vs Binary Tree

// This is a valid binary tree but NOT a BST:

```
      5
     / \
    3   8
     / \
    6   4  <- 4 is in right subtree of 5, but 4 < 5! VIOLATES BST!
```

[WARNING] **Professor Trap:** A binary tree is NOT a BST unless $\text{left} < \text{node} < \text{right}$ for ALL nodes!

2.5 Balanced Trees and AVL Trees

Why Balance Matters?

UNBALANCED ($O(n)$):

```
      1
       \
        2
         \
          3
           \
            4  <- Search is  $O(n)$ , not  $O(\log n)$ !
```

BALANCED ($O(\log n)$):

```
      2
     / \
    1   3
       \
        4  <- Search is  $O(\log n)$ 
```

AVL Trees

AVL tree is a self-balancing BST where heights of left and right subtrees differ by at most 1.

AVL Tree Properties:

- For every node: $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$
- Balance Factor = $\text{height}(\text{left}) - \text{height}(\text{right})$
- Balance factor must be -1, 0, or 1

Example AVL:

```
      10 (balance = 0)
     /  \
    5    15 (balance = 0)
```

Rotations

When a node becomes unbalanced, we rotate:

```
// Single Right Rotation (LL imbalance)
//
// Before:          After rotation:
//   z              y
//  /              / \
// y   T3   =>   x   z
// / \          /  / \
// x  T2        T1 T2 T3
//
TreeNode rightRotate(TreeNode z) {
    TreeNode y = z.left;
    TreeNode T2 = y.right;

    y.right = z;
    z.left = T2;

    // Update heights
    z.height = 1 + Math.max(height(z.left), height(z.right));
    y.height = 1 + Math.max(height(y.left), height(y.right));

    return y;
}

// Single Left Rotation (RR imbalance)
TreeNode leftRotate(TreeNode z) {
    TreeNode y = z.right;
    TreeNode T2 = y.left;

    y.left = z;
    z.right = T2;

    z.height = 1 + Math.max(height(z.left), height(z.right));
    y.height = 1 + Math.max(height(y.left), height(y.right));

    return y;
}
```

AVL Insert with Rebalancing

```
int getBalance(TreeNode node) {
    if (node == null) return 0;
    return height(node.left) - height(node.right);
}
```

```

TreeNode insert(TreeNode node, int key) {
    // 1. Standard BST insert
    if (node == null) return new TreeNode(key);

    if (key < node.data) node.left = insert(node.left, key);
    else if (key > node.data) node.right = insert(node.right, key);
    else return node;

    // 2. Update height
    node.height = 1 + Math.max(height(node.left), height(node.right));

    // 3. Get balance
    int balance = getBalance(node);

    // 4. Rebalance if needed

    // LL: Right rotate
    if (balance > 1 && key < node.left.data) {
        return rightRotate(node);
    }

    // RR: Left rotate
    if (balance < -1 && key > node.right.data) {
        return leftRotate(node);
    }

    // LR: Left then right
    if (balance > 1 && key > node.left.data) {
        node.left = leftRotate(node.left);
        return rightRotate(node);
    }

    // RL: Right then left
    if (balance < -1 && key < node.right.data) {
        node.right = rightRotate(node.right);
        return leftRotate(node);
    }

    return node;
}

```

Comparison: BST vs AVL

Aspect	BST	AVL
Balance	None	Self-maintained
Search	$O(h)$	$O(\log n)$ guaranteed

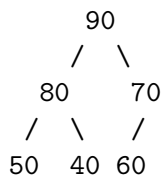
Aspect	BST	AVL
Insert/Delete	$O(h)$	$O(\log n)$ but extra rotations
Height	Can be $O(n)$	Always $O(\log n)$
Complexity	Simpler	More complex
When to use	When data is somewhat random	When search is frequent

2.6 Heaps

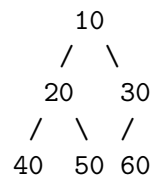
Definition / Theoretical Background

A **heap** is a complete binary tree that satisfies the heap property: - **Max Heap**: Parent $\geq$ children (largest at root) - **Min Heap**: Parent $\leq$ children (smallest at root)

MAX HEAP:



MIN HEAP:



Heap is COMPLETE: all levels filled, last level filled left to right

Heap as Array

```

// For index i:
// Parent: (i - 1) / 2
// Left child: 2 * i + 1
// Right child: 2 * i + 2

class MinHeap {
private int[] heap;
private int size;
private int capacity;

MinHeap(int capacity) {
    this.capacity = capacity;
    this.size = 0;
    heap = new int[capacity];
}

int parent(int i) { return (i - 1) / 2; }
int left(int i) { return 2 * i + 1; }
int right(int i) { return 2 * i + 2; }

// Heapify up (after insert)
void bubbleUp(int i) {

```

```

        while (i > 0 && heap[parent(i)] > heap[i]) {
            swap(i, parent(i));
            i = parent(i);
        }
    }

    // Heapify down (after delete root)
    void bubbleDown(int i) {
        while (left(i) < size) {
            int smallest = left(i);
            if (right(i) < size && heap[right(i)] < heap[left(i)]) {
                smallest = right(i);
            }
            if (heap[i] <= heap[smallest]) break;
            swap(i, smallest);
            i = smallest;
        }
    }
}

```

Heap Operations

```

class MinHeap {
    // Insert - O(log n)
    void insert(int value) {
        if (size == capacity) return;

        heap[size] = value;
        size++;
        bubbleUp(size - 1);
    }

    // Extract min - O(log n)
    int extractMin() {
        if (size == 0) throw new RuntimeException("Heap empty");

        int min = heap[0];
        heap[0] = heap[size - 1];
        size--;
        bubbleDown(0);
        return min;
    }

    // Get min - O(1)
    int getMin() {
        return heap[0];
    }
}

```

```

// Delete at index -  $O(\log n)$ 
void delete(int index) {
    decreaseKey(index, Integer.MIN_VALUE);
    extractMin();
}
}

```

Priority Queue using Heap

```

class PriorityQueue {
    private MinHeap heap;

    PriorityQueue() { heap = new MinHeap(100); }

    void enqueue(int priority, String value) {
        heap.insert(priority);
    }

    String dequeue() {
        return String.valueOf(heap.extractMin());
    }

    String peek() {
        return String.valueOf(heap.getMin());
    }

    boolean isEmpty() { return heap.size() == 0; }
}

```

Heap Applications

Application	Use Case
Priority Queue	Task scheduling, print queue
Heap Sort	$O(n \log n)$ sorting
Top K elements	Find largest K from N items
Merge K sorted arrays	Use min of all heads
Dijkstra's Algorithm	Select minimum distance vertex

Edge Cases & Traps

Trap 1: Array Index Calculation Error

```

// WRONG: Wrong formulas for parent/children
int parent(int i) { return i / 2; } // Wrong! Should be (i-1)/2
int left(int i) { return 2 * i; } // Wrong! Should be 2*i + 1

```

[WARNING] **Professor Trap:** For 0-indexed array: $\text{parent} = (i-1)/2$, $\text{left} = 2i+1$, $\text{right} = 2i+2$!

Trap 2: Forgetting to Update Size

```
// WRONG: Not incrementing size on insert
void insert(int value) {
    heap[size] = value;
    // size++; // FORGOT THIS!
    bubbleUp(size); // Wrong index!
}

// CORRECT:
void insert(int value) {
    heap[size] = value;
    bubbleUp(size); // Bubble up from size-1 after incrementing
    size++;
}
```

Actually correct implementation would bubble up from size-1, not size. But the key trap is size tracking.

Trap 3: Heap is NOT a BST!

```
// Heap property: parent > children (max heap)
// BST property: left subtree < node < right subtree

// This is a valid MAX HEAP but NOT a BST:
    50
   / \
  30  20    <- 30 > 20 but they're both < 50, okay for heap!
     / \
    10  15    <- 10 < 15 but in BST, left < right would matter!
```

[WARNING] **Professor Trap:** Heap only guarantees $\text{parent} > \text{children}$, not $\text{left} < \text{right}$!

Trap 4: Using Heap when BST is Better

```
// If you need to find ALL elements in sorted order:
// Heap is WRONG choice - use BST or sorting

// Heap can only get min/max efficiently
// BST can get elements in order via inorder traversal
```

[WARNING] **Professor Trap:** For sorted traversal, use BST not heap!

2.6 B+ Trees

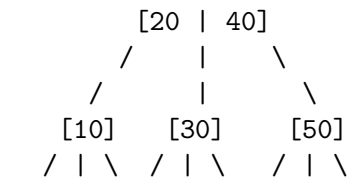
Definition / Theoretical Background

A **B+ Tree** is a self-balancing tree data structure that maintains sorted data in a way that allows efficient insertion, deletion, and search. It is a variant of B-Tree where all data (values) are stored

only at the leaf nodes, while internal nodes only contain keys for routing.

B+ TREE (order 3):

INTERNAL NODES:



<- (not actual children, just showing key ranges)

LEAF NODES:

```
[5 | 10 | 15] -> NULL
[25 | 30 | 35] -> NULL
[42 | 45 | 48] -> NULL
```

B+ Tree of order m:

- Each internal node has at most m children
- Each internal node (except root) has between m/2 and m children
- Root has at least 2 children (unless it's a leaf)
- All leaves are at the same level (height-balanced)
- All keys appear in sorted order in leaf nodes
- Leaf nodes are linked sequentially (allows range queries)

B+ Tree vs B-Tree vs BST

Aspect	BST	B-Tree	B+ Tree
Height	Can be $O(n)$	Logarithmic, very flat	Logarithmic, very flat
Storage	Each node stores one key	Internal nodes store keys AND values	Only leaf nodes store values
Range queries	$O(n)$ to traverse range	$O(n)$ to traverse range	$O(\log n + k)$ - leaves linked
Search	$O(\log n)$ average	$O(\log n)$	$O(\log n)$
Update	$O(\log n)$	$O(\log n)$	$O(\log n)$
Leaf linking	No	No	Yes - sequential access

Why B+ Trees are Used in Databases

B+ TREE ADVANTAGES FOR DISK:

1. FLAT STRUCTURE: Fewer disk reads to find data
 - BST with 1M keys, height ~20
 - B+ tree with 1M keys, height ~3-4
2. Fanout: Each node can store hundreds of keys
 - A node fits in one disk block (e.g., 4KB)
 - One disk read = many keys
3. LEAF LINKING: Efficient range queries
 - Find all keys between 10 and 50

- Start at 10, traverse linked leaves
- No need to go back up the tree

B+ Tree Node Structure

```
class BPlusNode {
    boolean isLeaf;
    int[] keys;           // Keys for routing (internal) or data (leaf)
    int keyCount;         // Number of keys currently stored
    BPlusNode[] children; // Child pointers (internal nodes only)
    BPlusNode next;       // Next leaf node (leaf nodes only, for linking)

    BPlusNode(int order, boolean isLeaf) {
        this.isLeaf = isLeaf;
        this.keys = new int[order - 1]; // Max keys = order - 1
        this.children = isLeaf ? null : new BPlusNode[order];
        this.next = null;
        this.keyCount = 0;
    }
}
```

B+ Tree Search

```
BPlusNode search(BPlusNode root, int key) {
    if (root == null) return null;

    if (root.isLeaf) {
        // Search in leaf node
        for (int i = 0; i < root.keyCount; i++) {
            if (root.keys[i] == key) {
                return root; // Found!
            }
        }
        return null; // Not found
    }

    // Internal node: find correct child
    int i = 0;
    while (i < root.keyCount && key >= root.keys[i]) {
        i++;
    }
    return search(root.children[i], key);
}
```

B+ Tree Insertion

```
void insert(BPlusNode root, int key, int value) {
    if (root == null) {
        return new BPlusNode(key, value);
    }

    // Find leaf node where key should go
    BPlusNode leaf = findLeaf(root, key);

    // Try to insert in leaf
    if (leaf.keyCount < MAX_KEYS) {
        insertIntoLeaf(leaf, key, value);
    } else {
        // Split leaf node
        BPlusNode newLeaf = splitLeaf(leaf);

        // Insert key into appropriate leaf
        if (key < leaf.keys[0]) {
            insertIntoLeaf(leaf, key, value);
        } else {
            insertIntoLeaf(newLeaf, key, value);
        }

        // Push middle key up to parent
        insertIntoParent(leaf, newLeaf.keys[0]);
    }
}
```

B+ Tree Split (Leaf Node)

```
BPlusNode splitLeaf(BPlusNode leaf) {
    BPlusNode newLeaf = new BPlusNode(true);

    int mid = leaf.keyCount / 2; // Or use (MAX_KEYS + 1) / 2

    // Copy second half to new leaf
    for (int i = mid; i < leaf.keyCount; i++) {
        newLeaf.keys[newLeaf.keyCount++] = leaf.keys[i];
    }

    // Link leaves
    newLeaf.next = leaf.next;
    leaf.next = newLeaf;

    leaf.keyCount = mid;
    return newLeaf;
}
```

```
}
```

B+ Tree Split (Internal Node)

```
BPlusNode splitInternal(BPlusNode node) {
    BPlusNode newNode = new BPlusNode(false);

    int mid = (node.keyCount - 1) / 2;  // Exclude middle key

    // Copy keys and children to new node
    for (int i = mid + 1; i < node.keyCount; i++) {
        newNode.keys[newNode.keyCount++] = node.keys[i];
    }

    // Children (excluding the one at mid)
    for (int i = mid + 1; i <= node.keyCount; i++) {
        newNode.children[newNode.keyCount] = node.children[i];
    }

    node.keyCount = mid;
    return newNode;
}
```

B+ Tree Delete

```
void delete(BPlusNode root, int key) {
    BPlusNode leaf = findLeaf(root, key);
    int index = findKeyInLeaf(leaf, key);

    if (index == -1) return;  // Key not found

    // Remove from leaf
    removeFromLeaf(leaf, index);

    // Check if leaf has too few keys (underflow)
    if (leaf.keyCount < MIN_KEYS) {
        // Try to borrow from sibling or merge
        BPlusNode sibling = findSibling(leaf);

        if (sibling.keyCount > MIN_KEYS) {
            borrowFromSibling(leaf, sibling);
        } else {
            mergeWithSibling(leaf, sibling);
        }
    }
}
```

B+ Tree Properties Summary

Property	Value
Order (m)	Maximum children per node
Min children	$\text{ceil}(m/2)$ for internal, $\text{ceil}(m/2)$ for leaf
Max keys per leaf	$m - 1$
Max keys per internal	$m - 1$
All leaves same depth	Yes
Leaves linked	Yes
Use for range query	Yes - $O(k)$ for k results

Edge Cases & Traps

Trap 1: Confusing B-Tree and B+ Tree

```
// B-Tree: Values stored in ALL nodes (internal and leaf)  
// B+ Tree: Values stored ONLY in leaf nodes  
  
// If professor asks "where are values stored in B+ tree?"  
// Answer: LEAF NODES ONLY!  
// Internal nodes only have keys for routing.
```

Trap 2: Order vs Max Keys

```
// If order m = 5:  
// - Max children = 5  
// - Max keys in leaf = 4 (not 5!)  
// - Max keys in internal = 4 (not 5!)  
  
// Formula: For order m:  
// - Max children = m  
// - Max keys = m - 1  
// - Min children = ceil(m/2) (internal)  
// - Min keys = ceil(m/2) - 1 (internal, except root)
```

[WARNING] **Professor Trap:** Order m means m children max, so $m-1$ keys max!

Trap 3: B+ Tree is Flat

```
// Why B+ tree has fewer levels than BST:  
// BST: 1 million nodes -> height ~20 ( $2^{20} = 1M$ )  
// B+ tree: 1 million nodes, order 100 -> height ~3  
// Level 0: 1 root  
// Level 1: 100 children  
// Level 2:  $100 * 100 = 10,000$  children  
// Level 3:  $10,000 * 100 = 1,000,000$  leaves  
  
// For disk-based storage, height 3 = only 3 disk reads vs 20!
```

[WARNING] **Professor Trap:** B+ tree's flat structure means fewer disk I/O operations!

2.7 Common Tree Operations Summary

Time Complexity Comparison

Operation	BST (avg)	BST (worst)	AVL	Heap
Search	$O(\log n)$	$O(n)$	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$	$O(\log n)$	$O(\log n)$
Delete	$O(\log n)$	$O(n)$	$O(\log n)$	$O(\log n)$
Find Min	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$
Find Max	$O(\log n)$	$O(n)$	$O(\log n)$	$O(1)$
Sorted Traversal	$O(n)$	$O(n)$	$O(n)$	$O(n \log n)^*$

*Heap would need to extract all elements

Space Complexity

All tree structures: $O(n)$ where n is number of nodes

Tree Selection Guide

Need	Use
Fast search	BST or AVL
Sorted output	BST
Guaranteed $O(\log n)$	AVL
Priority queue	Heap
Top K elements	Min-Heap
Auto-balancing	AVL, Red-Black Tree

2.8 Practice Questions

MCQ 1: BST Search

What is the time complexity of searching in a balanced BST with n nodes? - A) $O(1)$ - B) $O(n)$ - C) $O(\log n)$ - D) $O(n \log n)$

MCQ 2: Tree Traversal

Which traversal visits root before left and right children? - A) Inorder - B) Postorder - C) Preorder - D) Level order

MCQ 3: AVL Trees

What is the maximum height difference between left and right subtrees in an AVL tree? - A) 0 - B) 1 - C) 2 - D) Any value

MCQ 4: Heap Property

In a max heap, which node has the largest value? - A) Any leaf - B) Root - C) Leftmost node - D) Rightmost node

Short Answer 1

Explain why AVL trees require rotations after insertions while regular BSTs don't.

Short Answer 2

What is the advantage of heap over BST for implementing priority queue?

Board Coding 1

Implement inorder traversal of a BST iteratively (using stack, not recursion).

Board Coding 2

Write a function to check if a binary tree is a valid BST.

Board Coding 3

Implement heap sort: given an array, sort it using a max heap.

Oral Explanation

Compare complete binary tree vs full binary tree vs perfect binary tree with diagrams.

Solutions

MCQ 1: C) $O(\log n)$ - Balanced BST has height $\sim \log n$

MCQ 2: C) Preorder - Visits root first (N-L-R)

MCQ 3: B) 1 - AVL balance factor must be -1, 0, or 1

MCQ 4: B) Root - Max heap: root is always maximum

Short Answer 1: AVL trees maintain strict balance (height diff ≤ 1). When inserting, if balance factor exceeds 1, we rotate to restore balance. Regular BST has no balance constraint, so it can become degenerate without correction.

Short Answer 2: Heap gives $O(1)$ for getMin/getMax vs $O(\log n)$ for BST. Priority queues need constant time access to highest/lowest priority element, making heap more efficient.

Cheat-Sheet

Traversal Orders

Preorder: Root $\rightarrow$ Left $\rightarrow$ Right

Inorder: Left $\rightarrow$ Root $\rightarrow$ Right

Postorder: Left -> Right -> Root
 Level: Level by level (BFS)

BST vs AVL vs Heap

Property	BST	AVL	Heap
Shape	Any	Balanced	Complete
Order	Left < Root < Right	Left < Root < Right	Root > Children
Search	$O(\log n)$ avg	$O(\log n)$	$O(n)$
Min/Max	$O(\log n)$	$O(\log n)$	$O(1)$
Insert	$O(\log n)$ avg	$O(\log n)$	$O(\log n)$

Review Checklist

- ☐ **Tree fundamentals:** Can you define root, leaf, parent, child, depth, height?
- ☐ **Binary tree types:** Can you distinguish full, perfect, complete, balanced?
- ☐ **Traversals:** Can you implement all 4 traversals (recursive and iterative)?
- ☐ **BST:** Can you implement search, insert, delete? Do you know the 3 delete cases?
- ☐ **AVL:** Do you understand balance factors and when rotations are needed?
- ☐ **Heap:** Can you implement insert and extract? Do you know array indexing?
- ☐ **When to use which:** Can you choose the right structure for given problem?
- ☐ **Traps:** Null checks, rotation types, duplicate handling, array indexing formulas? # Chapter 3: Hashing Techniques

3.1 Introduction to Hashing

Definition / Theoretical Background

Hashing is a technique to map data of arbitrary size to fixed-size values using a **hash function**. A **hash table** stores key-value pairs and provides $O(1)$ average-case lookup, insertion, and deletion.

HASH TABLE CONCEPT:

Key (input)	Hash Function	Index	Table
-----	----->	-----	-----
"apple"	--hash("apple")-->	2	0
			1
"banana"	--hash("banana")-->	0	2 "apple" -> "fruit"
			3
"cherry"	--hash("cherry")-->	1	4

Direct Access: hash("apple") -> index 2 -> $O(1)$ lookup!

Why Hashing?

Data Structure	Search	Insert	Delete	Ordered?
Array (unsorted)	$O(n)$	$O(1)^*$	$O(n)$	No

Data Structure	Search	Insert	Delete	Ordered?
Array (sorted)	$O(\log n)$	$O(n)$	$O(n)$	Yes
Linked List	$O(n)$	$O(1)^*$	$O(1)^*$	No
BST (balanced)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No

*At end of structure

Hash Table vs BST

Aspect	Hash Table	BST
Search complexity	$O(1)$ average	$O(\log n)$
Order	No ordering	Sorted (inorder)
Worst case	$O(n)$	$O(\log n)$
Memory	Extra for empty slots	Compact
Implementation	Simpler	More complex
Use when	Need fast lookup, don't need order	Need sorted data

3.2 Hash Functions

Definition

A **hash function** converts a key into an index in the hash table.

Properties of Good Hash Function:

1. Deterministic: Same key always produces same hash
2. Uniform distribution: Hashes spread uniformly across table
3. Fast: $O(1)$ computation
4. Minimize collisions: Different keys should map to different indices when possible

Common Hash Functions

```
int hash(int key, int tableSize) {
    return key % tableSize;
}
```

```
// Examples with tableSize = 10:
// hash(25) = 25 % 10 = 5
// hash(127) = 127 % 10 = 7
// hash(5432) = 5432 % 10 = 2
```

1. Division Method (Modular Arithmetic)

```
int hash(int key, int tableSize) {
    double A = 0.6180339887; // Golden ratio conjugate
    double fractional = (key * A) % 1.0;
    return (int)(tableSize * fractional);
}

// Formula: floor(m * (key * A mod 1))
// Where A is constant (often golden ratio)
```

2. Multiplication Method

```
// Simple string hash
int hashString(String s, int tableSize) {
    int hash = 0;
    for (char c : s.toCharArray()) {
        hash = (hash * 31 + c) % tableSize;
    }
    return hash;
}

// Why 31? Prime number that works well for string hashing
// Java's String.hashCode() uses similar approach

// Examples with tableSize = 100:
// hashString("apple", 100) = ?
// hashString("banana", 100) = ?
```

3. String Hashing

Hash Code Implementation in Java

```
// Default Object.hashCode() implementation
// Usually based on memory address

// String hashCode (Java)
public int hashCode() {
    int h = 0;
    for (int i = 0; i < length(); i++) {
        h = 31 * h + charAt(i);
    }
    return h;
}

// Integer uses value itself
// .hashCode() = Integer.hashCode(42) = 42
```

```
// For custom objects, override hashCode()
class Person {
    String name;
    int age;

    @Override
    public int hashCode() {
        return name.hashCode() + age;
    }
}
```

Good Table Size

```
// Table size should be:
// 1. Prime number (reduces clustering)
// 2. Not power of 2 (causes poor bit distribution)
// 3. Large enough to keep load factor low

// Bad choices:
// tableSize = 2^k (easy to hash but poor distribution)
// tableSize = 100 (not prime)

// Good choices:
// tableSize = 7, 13, 29, 53, 97, 193, 389, 769, ...
// tableSize = 1009, 2003, 5003, 10007, ...
```

3.3 Collisions

Definition

A **collision** occurs when two different keys hash to the same index.

COLLISION EXAMPLE:

```
hash("apple") = 2
hash("avocado") = 2    <- COLLISION! Both map to index 2
```

Table[2] can only hold ONE value!

Collisions are INEVITABLE (pigeonhole principle):

If n items and k slots, and $n > k$, at least one slot has multiple items

Collision Resolution Strategies

Strategy	How it works	Pros	Cons
Chaining	Linked list at each slot	Simple, handles unlimited items	Memory overhead
Open Addressing	Find next empty slot	No extra memory	Limited items, clustering issues

3.4 Chaining (Separate Chaining)

Implementation

Each slot contains a linked list of items that hash to that index.

```
class HashNode {
    String key;
    String value;
    HashNode next;

    HashNode(String key, String value) {
        this.key = key;
        this.value = value;
        this.next = null;
    }
}

class HashTable {
    private HashNode[] chainArray;
    private int tableSize;
    private int size;

    HashTable(int tableSize) {
        this.tableSize = tableSize;
        this.chainArray = new HashNode[tableSize];
        this.size = 0;
    }

    int hash(String key) {
        int hash = key.hashCode() % tableSize;
        return Math.abs(hash); // Ensure non-negative
    }

    // Insert - O(1) average
    void insert(String key, String value) {
        int index = hash(key);
```

```

// Check if key already exists, update if so
HashNode current = chainArray[index];
while (current != null) {
    if (current.key.equals(key)) {
        current.value = value;
        return;
    }
    current = current.next;
}

// Insert at head of chain
HashNode newNode = new HashNode(key, value);
newNode.next = chainArray[index];
chainArray[index] = newNode;
size++;
}

// Get - O(1) average
String get(String key) {
    int index = hash(key);

    HashNode current = chainArray[index];
    while (current != null) {
        if (current.key.equals(key)) {
            return current.value;
        }
        current = current.next;
    }
    return null; // Key not found
}

// Delete - O(1) average
boolean delete(String key) {
    int index = hash(key);

    HashNode current = chainArray[index];
    HashNode prev = null;

    while (current != null) {
        if (current.key.equals(key)) {
            if (prev == null) {
                chainArray[index] = current.next;
            } else {
                prev.next = current.next;
            }
            size--;
            return true;
        }
    }
}

```

```

        }
        prev = current;
        current = current.next;
    }
    return false;
}

boolean contains(String key) {
    return get(key) != null;
}

int size() { return size; }

boolean isEmpty() { return size == 0; }
}

```

Chaining Diagram

CHAINING VISUALIZATION (tableSize = 5):

```

Index 0: [ "john" -> "NYC" ] -> null
Index 1: null
Index 2: [ "apple" -> "fruit" ] -> [ "avocado" -> "food" ] -> null
Index 3: [ "cat" -> "animal" ] -> null
Index 4: null

```

hash("apple") = 2, hash("avocado") = 2
Both stored in chain at index 2

Load Factor and Rehashing

```

double loadFactor() {
    return (double) size / tableSize;
}

// Rehashing when load factor exceeds threshold (usually 0.75)

void rehash() {
    HashNode[] oldArray = chainArray;
    int oldSize = tableSize;

    tableSize = tableSize * 2;
    chainArray = new HashNode[tableSize];
    size = 0;

    // Reinsert all items
    for (int i = 0; i < oldSize; i++) {

```

```

        HashNode current = oldArray[i];
        while (current != null) {
            insert(current.key, current.value);
            current = current.next;
        }
    }
}

// When loadFactor > 0.75, call rehash()

```

Time Complexity with Chaining

Operation	Average Case	Worst Case
Search	$O(1 + \text{load factor})$	$O(n)$
Insert	$O(1)$	$O(n)$
Delete	$O(1 + \text{load factor})$	$O(n)$
Contains	$O(1 + \text{load factor})$	$O(n)$

3.5 Open Addressing

Definition

Open addressing stores all items in the table itself. When collision occurs, find another open slot in the table.

OPEN ADDRESSING:

If $\text{hash}(\text{"apple"}) = 2$ but slot 2 is full:

- Try slot 3, then 4, then 5... (linear probing)
- Or use quadratic probing
- Or use double hashing

All items stored IN the table, not in external chains

Types of Open Addressing

Type	Collision Resolution
Linear Probing	Check next slot sequentially
Quadratic Probing	Check slots with quadratic jumps
Double Hashing	Use second hash function

3.6 Linear Probing

Implementation

```
class LinearProbingHashTable {
    private String[] keys;
    private String[] values;
    private int tableSize;
    private int size;
    private final String DELETED = "DELETED";

    LinearProbingHashTable(int tableSize) {
        this.tableSize = tableSize;
        this.keys = new String[tableSize];
        this.values = new String[tableSize];
        this.size = 0;
    }

    int hash(String key) {
        return Math.abs(key.hashCode() % tableSize);
    }

    // Probe: try consecutive slots
    int probe(int index) {
        return (index + 1) % tableSize;
    }

    void insert(String key, String value) {
        if (size >= tableSize) {
            throw new RuntimeException("Table full");
        }

        int index = hash(key);

        // Find empty slot
        while (keys[index] != null && !keys[index].equals(DELETED)) {
            if (keys[index].equals(key)) {
                values[index] = value; // Update existing
                return;
            }
            index = probe(index); // Move to next slot
        }

        keys[index] = key;
        values[index] = value;
        size++;
    }
}
```

```

String get(String key) {
    int index = hash(key);

    while (keys[index] != null) {
        if (keys[index].equals(key)) {
            return values[index];
        }
        index = probe(index);
    }
    return null;
}

boolean delete(String key) {
    int index = hash(key);

    while (keys[index] != null) {
        if (keys[index].equals(key)) {
            keys[index] = DELETED; // Mark as deleted
            values[index] = null;
            size--;
            return true;
        }
        index = probe(index);
    }
    return false;
}
}

```

Linear Probing Example

Insert: 22, 35, 42 with tableSize = 7

hash(22) = $22 \% 7 = 1$ -> slot 1 [empty] -> insert at 1

hash(35) = $35 \% 7 = 0$ -> slot 0 [empty] -> insert at 0

hash(42) = $42 \% 7 = 0$ -> slot 0 [full!] -> try 1, also full! -> try 2 [empty] -> insert at 2

Final table:

Index:	0	1	2	3	4	5	6
	35	22	42	-	-	-	-

Primary Clustering Problem

PRIMARY CLUSTERING:

Once a contiguous block forms, new items tend to extend it
 Long chains grow longer, increasing search time

Example:

- Items at indices 3, 4, 5, 6 form cluster
 - New item that hashes to 3 will probe through all of them
 - This makes big clusters grow bigger (positive feedback)
-

3.7 Quadratic Probing

Implementation

```
class QuadraticProbingHashTable {
    private String[] keys;
    private String[] values;
    private int tableSize;
    private int size;
    private final String DELETED = "DELETED";

    QuadraticProbingHashTable(int tableSize) {
        this.tableSize = tableSize;
        this.keys = new String[tableSize];
        this.values = new String[tableSize];
        this.size = 0;
    }

    int hash(String key) {
        return Math.abs(key.hashCode() % tableSize);
    }

    // Quadratic probe: 1^2, 2^2, 3^2, ...
    int probe(int index, int i) {
        return (index + i * i) % tableSize;
    }

    void insert(String key, String value) {
        int index = hash(key);
        int i = 0;

        while (keys[index] != null && !keys[index].equals(DELETED)) {
            if (keys[index].equals(key)) {
                values[index] = value;
                return;
            }
            i++;
            index = probe(index, i);
        }

        keys[index] = key;
        values[index] = value;
    }
}
```

```

        size++;
    }

    String get(String key) {
        int index = hash(key);
        int i = 0;

        while (keys[index] != null) {
            if (keys[index].equals(key)) {
                return values[index];
            }
            i++;
            index = probe(index, i);
        }
        return null;
    }
}

```

Quadratic Probing Example

Insert: 22, 35, 42 with tableSize = 7

hash(22) = 1 -> slot 1 [empty] -> insert at 1

hash(35) = 0 -> slot 0 [empty] -> insert at 0

hash(42) = 0 -> slot 0 [full] -> try $(0 + 1^2) = 1$, also full! -> try $(0 + 2^2) = 4$ [empty] ->

Final table:

Index:	0	1	2	3	4	5	6
	35	22	-	-	42	-	-

Secondary Clustering Problem

SECONDARY CLUSTERING:

Items that hash to same initial slot probe the same sequence

Two items with same initial hash will follow same probe sequence

Example:

- Item A hashes to index 3, collides
- Item B also hashes to index 3, collides
- Both will probe indices 4, 7, 12, ...
- Even though A and B are different items!

3.8 Double Hashing

Implementation

```
class DoubleHashingHashTable {
    private String[] keys;
    private String[] values;
    private int tableSize;
    private int size;
    private final String DELETED = "DELETED";

    DoubleHashingHashTable(int tableSize) {
        this.tableSize = tableSize;
        this.keys = new String[tableSize];
        this.values = new String[tableSize];
        this.size = 0;
    }

    int hash1(String key) {
        return Math.abs(key.hashCode() % tableSize);
    }

    // Second hash function must never return 0
    int hash2(String key) {
        return 5 - (Math.abs(key.hashCode()) % 5);
    }

    int probe(int index, int i) {
        int h2 = hash2(index); // Use index as key for h2
        return (index + i * h2) % tableSize;
    }

    void insert(String key, String value) {
        int index = hash1(key);
        int i = 0;

        while (keys[index] != null && !keys[index].equals(DELETED)) {
            if (keys[index].equals(key)) {
                values[index] = value;
                return;
            }
            i++;
            index = probe(index, i);
        }

        keys[index] = key;
        values[index] = value;
        size++;
    }
}
```

```

    }

    String get(String key) {
        int index = hash1(key);
        int i = 0;

        while (keys[index] != null) {
            if (keys[index].equals(key)) {
                return values[index];
            }
            i++;
            index = probe(index, i);
        }
        return null;
    }
}

```

Double Hashing Formula

Double hashing probe sequence:

$$h(\text{key}, i) = (h1(\text{key}) + i * h2(\text{key})) \% \text{tableSize}$$

Where:

- $h1(\text{key}) = \text{key} \% \text{tableSize}$ (primary hash)
- $h2(\text{key}) = \text{prime} - (\text{key} \% \text{prime})$ (secondary hash, $\text{prime} < \text{tableSize}$)

Example:

- $\text{tableSize} = 7$
- $h1(\text{"apple"}) = 2$
- $h2(\text{"apple"}) = 5 - (2 \% 5) = 3$
- Probes: 2, $(2+1*3)\%7=5$, $(2+2*3)\%7=1$, $(2+3*3)\%7=4$, ...

3.9 Comparison of Collision Resolution Methods

Method	How it Works	Pros	Cons	When to Use
Chaining	Linked list at each slot	Simple, handles many items	Extra memory for lists	Unknown number of items
Linear Probing	Sequential search	Simple, good cache locality	Primary clustering	Low load factor
Quadratic Probing	Quadratic jumps	Avoids primary clustering	Secondary clustering	Medium load factor

Method	How it Works	Pros	Cons	When to Use
Double Hashing	Second hash function	Best clustering avoidance	More computations	High load factor

Time Complexity Comparison

Method	Average Search	Worst Search	Insert	Delete
Chaining	$O(1 + \text{load factor})$	$O(n)$	$O(1)$	$O(1 + \text{load factor})$
Linear Probing	$O(1 / (1 - \text{load factor}))$	$O(n)$	$O(1 / (1 - \text{load factor}))$	$O(n)$
Quadratic Probing	$O(1 / (1 - \text{load factor}))$	$O(n)$	$O(1 / (1 - \text{load factor}))$	$O(n)$
Double Hashing	$O(1 / (1 - \text{load factor}))$	$O(n)$	$O(1 / (1 - \text{load factor}))$	$O(n)$

Load Factor Guidelines

Load Factor	Recommendation
< 0.5	Good performance
$0.5 - 0.75$	Acceptable, consider rehashing
> 0.75	High collision risk, REHASH!
> 0.9	Very poor performance

3.10 HashMap in Java

Built-in Implementation

```
import java.util.HashMap;

// Basic operations
HashMap<String, Integer> map = new HashMap<>();

// Insert - O(1)
map.put("apple", 1);
map.put("banana", 2);
map.put("cherry", 3);

// Get - O(1)
Integer val = map.get("banana"); // Returns 2
Integer missing = map.get("grape"); // Returns null
```

```

// Check existence
boolean hasApple = map.containsKey("apple"); // true
boolean hasValue2 = map.containsValue(2);    // true

// Remove - O(1)
map.remove("banana");

// Size
int size = map.size(); // 2

// Iterate
for (String key : map.keySet()) {
    System.out.println(key + ": " + map.get(key));
}

// ForEach (Java 8+)
map.forEach((k, v) -> System.out.println(k + ": " + v));

// Get or default
int count = map.getDefault("apple", 0); // Returns 1 (exists)
int newCount = map.getDefault("grape", 0); // Returns 0 (not exists)

```

HashMap Performance

```

HashMap<String, Integer> map = new HashMap<>(16, 0.75f);
// Initial capacity: 16
// Load factor: 0.75 (rehash when 75% full)

// Operations:
// put, get, remove: O(1) average
// containsKey: O(1) average
// keySet, values, entrySet: O(n)

```

HashMap vs Hashtable

Aspect	HashMap	Hashtable
Thread safety	Not synchronized	Synchronized
Null keys	Allows one null key	No null keys
Null values	Allows multiple null values	No null values
Performance	Faster (not thread-safe)	Slower (thread-safe)
Legacy	New (Java 1.2)	Legacy (original)

3.11 HashSet in Java

Implementation

```
import java.util.HashSet;

// HashSet stores unique elements (no duplicates)
HashSet<String> set = new HashSet<>();

// Add - O(1)
set.add("apple");
set.add("banana");
set.add("apple"); // No effect (already exists)

// Contains - O(1)
boolean hasBanana = set.contains("banana"); // true
boolean hasGrape = set.contains("grape");   // false

// Remove - O(1)
set.remove("banana");

// Size
int size = set.size(); // 1 (only "apple" left)

// Iterate
for (String s : set) {
    System.out.println(s);
}

// Common operations
set.addAll(otherSet); // Union
set.retainAll(otherSet); // Intersection
set.removeAll(otherSet); // Difference
```

3.12 Hashing Applications

1. Caching / Memoization

```
// Fibonacci with memoization
class FibMemo {
    private Map<Integer, Long> cache = new HashMap<>();

    long fib(int n) {
        if (cache.containsKey(n)) return cache.get(n);
        if (n <= 1) return n;

        long result = fib(n - 1) + fib(n - 2);
    }
}
```

```

        cache.put(n, result);
        return result;
    }
}

```

2. Counting Unique Elements

```

// Count unique words in text
Set<String> uniqueWords = new HashSet<>();
for (String word : text.split("\\s+")) {
    uniqueWords.add(word.toLowerCase());
}
System.out.println("Unique words: " + uniqueWords.size());

```

3. Lookup Tables

```

// IP to country lookup
Map<String, String> ipCountry = new HashMap<>();
ipCountry.put("192.168.1.1", "USA");
ipCountry.put("10.0.0.1", "Private");

String getCountry(String ip) {
    return ipCountry.getOrDefault(ip, "Unknown");
}

```

4. De-duplication

```

// Remove duplicates from list
List<Integer> list = Arrays.asList(1, 2, 3, 2, 1, 4, 3);
Set<Integer> unique = new HashSet<>(list);
List<Integer> deduped = new ArrayList<>(unique);

```

3.13 Edge Cases & Traps

Trap 1: Hash Table Full (Open Addressing)

```

// If table is full, insert loops forever!
void insert(String key, String value) {
    int index = hash(key);
    while (keys[index] != null) {
        index = probe(index);
        // If table is FULL, infinite loop!
    }
}

```

```
// Solution: Check if size < tableSize before insert
```

Trap 2: Using Mutable Keys

```
// WRONG: Using mutable object as key
```

```
class Person {  
    String name;  
    int age;  
    // No hashCode override!  
}  
  
Person p = new Person("John", 25);  
map.put(p, "developer");  
p.age = 30; // Changed hashCode!  
// Now map.get(p) may fail!
```

[WARNING] **Professor Trap:** Keys should be immutable or use defensive copying!

Trap 3: Integer Overflow in Hash Function

```
// Integer overflow example  
int hash = key.hashCode() % tableSize;  
// If key.hashCode() is Integer.MIN_VALUE, Math.abs also returns MIN_VALUE!  
// Math.abs(Integer.MIN_VALUE) = Integer.MIN_VALUE (still negative!)  
  
int index = Math.abs(key.hashCode()) % tableSize; // Could still be negative!
```

[WARNING] **Professor Trap:** Always use Math.abs carefully, or use long for hash calculations!

Trap 4: String Hashing Inefficiency

```
// Bad: Creating new string for each character  
public int hashCode() {  
    int h = 0;  
    for (int i = 0; i < length(); i++) {  
        h = (h * 31 + charAt(i)) % tableSize; // Expensive mod each time!  
    }  
    return h;  
}  
  
// Better: Only mod at end  
public int hashCode() {  
    int h = 0;  
    for (int i = 0; i < length(); i++) {  
        h = h * 31 + charAt(i);  
    }  
    return h % tableSize;  
}
```

Trap 5: Rehashing with Iterators

```

// Concurrent modification during iteration
for (String key : map.keySet()) {
    if (key.equals("bad")) {
        map.remove(key); // ConcurrentModificationException!
    }
}

// Fix: Use Iterator
Iterator<String> it = map.keySet().iterator();
while (it.hasNext()) {
    String key = it.next();
    if (key.equals("bad")) {
        it.remove();
    }
}

```

Trap 6: Wrong Table Size

```

// Bad: Table size of 2^k with power-of-2 hashing
// If using (key % tableSize) with tableSize = 16 = 2^4
// Only lower 4 bits of key affect index!

// Good: Prime table size
tableSize = 7, 13, 31, 61, 127, ...

```

3.14 Practice Questions

MCQ 1: Hash Collision

Two different keys produce the same hash value. This is called: - A) Overloading - B) Collision - C) Overflow - D) Redundancy

MCQ 2: Hash Table Average Search

What is the average time complexity for searching in a hash table? - A) $O(1)$ - B) $O(n)$ - C) $O(\log n)$ - D) $O(n \log n)$

MCQ 3: Collision Resolution

Which is NOT an open addressing technique? - A) Linear probing - B) Quadratic probing - C) Chaining - D) Double hashing

MCQ 4: Load Factor

If a hash table has 100 slots and 75 elements, the load factor is: - A) 0.25 - B) 0.75 - C) 1.33 - D) 1.0

Short Answer 1

Explain why hash tables are faster than BSTs for lookups despite having $O(n)$ worst-case complexity.

Short Answer 2

What is primary clustering in linear probing and why is it a problem?

Board Coding 1

Implement a hash table with chaining (separate chaining) including insert, get, and delete operations.

Board Coding 2

Write a function to find the first non-repeating character in a string using hashing.

Oral Explanation

Compare chaining and open addressing. In what scenarios would you choose one over the other?

Solutions

MCQ 1: B) Collision - Two keys mapping to same index

MCQ 2: A) $O(1)$ average - Constant time with good hash function

MCQ 3: C) Chaining - Chaining is separate chaining, not open addressing

MCQ 4: B) 0.75 - Load factor = $n/m = 75/100 = 0.75$

Short Answer 1: Hash tables provide $O(1)$ average because the hash function directly computes the index, requiring only one table lookup. BSTs require traversing from root to leaf, which takes $O(\log n)$ comparisons even in balanced case. The constant factor is smaller for hash tables.

Short Answer 2: Primary clustering occurs when items that hash to nearby indices form long contiguous sequences. This causes new items to probe through these chains, making them grow even longer. Results in poor performance for large clusters.

Cheat-Sheet

Hash Function Properties

- Deterministic (same key $\rightarrow$ same hash)
- Uniform distribution
- $O(1)$ computation
- Minimize collisions

Collision Resolution

Method	Collision Strategy	Memory	Performance
Chaining	Linked lists	Extra for lists	Good worst case
Linear Probing	Sequential	Compact	Primary clustering
Quadratic Probing	Quadratic jumps	Compact	Secondary clustering
Double Hashing	Second hash	Compact	Best distribution

Load Factor

- Load factor > 0.75 : REHASH!
- Keep load factor low for better performance

Time Complexity (Average Case)

Operation	Hash Table
Search	$O(1)$
Insert	$O(1)$
Delete	$O(1)$
Get keys	$O(n)$

Review Checklist

- ☐ **Hash functions:** Can you implement division and multiplication methods?
- ☐ **Collisions:** Do you understand why collisions are inevitable?
- ☐ **Chaining:** Can you implement insert, get, delete with chaining?
- ☐ **Open addressing:** Can you explain linear, quadratic, and double hashing?
- ☐ **Load factor:** Do you know when to rehash?
- ☐ **Java HashMap:** Can you use get, put, remove, containsKey?
- ☐ **Traps:** Are you aware of clustering, mutable keys, and table size issues?
- ☐ **When to use:** Can you decide between hash table vs BST vs array? # Chapter 4: Graphs

4.1 Introduction to Graphs

Definition / Theoretical Background

A **graph** is a non-linear data structure consisting of vertices (nodes) connected by edges. Unlike trees, graphs can have cycles and multiple paths between nodes.

GRAPH STRUCTURE:

Undirected Graph:

```
  A --- B
  / |   |
 /  |   |
C  +---- D
```

Directed Graph (Digraph):

```
  A --> B
  |     ^
  v     |
  C --> D
```

Vertices: A, B, C, D

Edges: (A,B), (A,C), (B,D), (C,D)

Graph Terminology

Term	Definition	Example
Vertex (Node)	A point in the graph	A, B, C, D
Edge	Connection between vertices	(A, B), (B, C)
Adjacent vertices	Connected by an edge	A and B are adjacent

Term	Definition	Example
Degree	Number of edges at a vertex	Degree of A = 2
In-degree	Edges coming INTO vertex (digraph)	In-degree of B = 1
Out-degree	Edges going OUT of vertex (digraph)	Out-degree of A = 1
Path	Sequence of vertices	A -> C -> D
Cycle	Path that returns to starting vertex	A -> B -> D -> A
Connected	Path exists between all vertex pairs	Entire graph reachable

Types of Graphs

Type	Description	Example
Undirected	Edges have no direction	Friendship on Facebook
Directed (Digraph)	Edges have direction	Twitter follow
Weighted	Edges have weights/costs	Road distances
Unweighted	All edges equal weight	Social network
Cyclic	Contains cycles	
Acyclic	No cycles	DAG (Directed Acyclic Graph)
Complete	All pairs of vertices connected	
Sparse	Few edges ($E \ll V^2$)	
Dense	Many edges (E close to V^2)	

4.2 Graph Representations

1. Adjacency Matrix

2D array where $\text{matrix}[i][j] = \text{weight}$ if edge exists, else 0 or infinity

```
class GraphMatrix {
    private int[][] matrix;
    private int numVertices;
    private boolean directed;

    GraphMatrix(int numVertices, boolean directed) {
        this.numVertices = numVertices;
        this.directed = directed;
        matrix = new int[numVertices][numVertices];

        // Initialize with 0 for no edge (unweighted) or INF for weighted
        for (int i = 0; i < numVertices; i++) {
            for (int j = 0; j < numVertices; j++) {
                if (i == j) matrix[i][j] = 0; // No self-loops
                else matrix[i][j] = Integer.MAX_VALUE; // No edge
            }
        }
    }
}
```

```

void addEdge(int v1, int v2) {
    addEdge(v1, v2, 1); // Default weight = 1
}

void addEdge(int v1, int v2, int weight) {
    matrix[v1][v2] = weight;
    if (!directed) {
        matrix[v2][v1] = weight; // Undirected: symmetric
    }
}

void removeEdge(int v1, int v2) {
    matrix[v1][v2] = Integer.MAX_VALUE;
    if (!directed) matrix[v2][v1] = Integer.MAX_VALUE;
}

boolean hasEdge(int v1, int v2) {
    return matrix[v1][v2] != Integer.MAX_VALUE;
}

void print() {
    for (int i = 0; i < numVertices; i++) {
        for (int j = 0; j < numVertices; j++) {
            if (matrix[i][j] == Integer.MAX_VALUE) System.out.print("INF ");
            else System.out.print(matrix[i][j] + " ");
        }
        System.out.println();
    }
}

List<Integer> getNeighbors(int vertex) {
    List<Integer> neighbors = new ArrayList<>();
    for (int j = 0; j < numVertices; j++) {
        if (matrix[vertex][j] != Integer.MAX_VALUE && vertex != j) {
            neighbors.add(j);
        }
    }
    return neighbors;
}
}

```

Adjacency Matrix Example

GRAPH:

```

A --- B
|     |
|     |

```

ADJACENCY MATRIX:

	A	B	C	D
A	0	1	1	INF
B	1	0	0	1

C	---	D				
			C	1	0	0
			D	INF	1	1
					0	0

2. Adjacency List

Array of lists where each list contains neighbors of a vertex

```
class GraphList {
    private List<Integer>[] adjList;
    private int numVertices;
    private boolean directed;

    GraphList(int numVertices, boolean directed) {
        this.numVertices = numVertices;
        this.directed = directed;
        adjList = new ArrayList[numVertices];
        for (int i = 0; i < numVertices; i++) {
            adjList[i] = new ArrayList<>();
        }
    }

    void addEdge(int v1, int v2) {
        adjList[v1].add(v2);
        if (!directed) {
            adjList[v2].add(v1);
        }
    }

    void addEdgeWeighted(int v1, int v2, int weight) {
        adjList[v1].add(v2); // Store weight separately if needed
        if (!directed) {
            adjList[v2].add(v1);
        }
    }

    void removeEdge(int v1, int v2) {
        adjList[v1].remove(Integer.valueOf(v2));
        if (!directed) {
            adjList[v2].remove(Integer.valueOf(v1));
        }
    }

    List<Integer> getNeighbors(int vertex) {
        return adjList[vertex];
    }

    void print() {
        for (int i = 0; i < numVertices; i++) {
```

```

        System.out.print(i + " -> ");
        for (int neighbor : adjList[i]) {
            System.out.print(neighbor + " ");
        }
        System.out.println();
    }
}

```

Adjacency List Example

GRAPH:	ADJACENCY LIST:
A --> B	A: B, C
	B: D
v v	C: D
C --> D	D: (empty)

Matrix vs List Comparison

Aspect	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Add edge	$O(1)$	$O(1)$
Remove edge	$O(1)$	$O(V)$ (need to search)
Check edge	$O(1)$	$O(V)$
Find neighbors	$O(V)$	$O(\text{degree})$
Dense graph	Better	Worse
Sparse graph	Wasteful	Better
Cache friendliness	Better	Worse

Use matrix when: - Graph is dense ($E \sim V^2$) - Need $O(1)$ edge lookup - Need to quickly check if edges exist

Use list when: - Graph is sparse ($E \ll V^2$) - Need to iterate neighbors efficiently - Memory is limited

4.3 Breadth-First Search (BFS)

Definition

BFS explores vertices in “layers” - visits all neighbors before going deeper. Uses a **queue** data structure.

BFS on Graph:

```

    0
   /\
  1 2 3

```

```
  | | |
  4 5 6
```

BFS Order (starting from 0):

Level 0: 0

Level 1: 1, 2, 3

Level 2: 4, 5, 6

Queue visualization:

Step 1: enqueue(0) -> Queue: [0]

Step 2: dequeue(0), enqueue neighbors(1,2,3) -> Queue: [1,2,3]

Step 3: dequeue(1), enqueue neighbors(4) -> Queue: [2,3,4]

Step 4: dequeue(2), enqueue neighbors(5) -> Queue: [3,4,5]

Step 5: dequeue(3), enqueue neighbors(6) -> Queue: [4,5,6]

Step 6: dequeue(4) -> Queue: [5,6]

...

BFS Implementation

```
class GraphBFS {
    private List<Integer>[] adjList;
    private int numVertices;

    GraphBFS(int numVertices) {
        this.numVertices = numVertices;
        adjList = new ArrayList[numVertices];
        for (int i = 0; i < numVertices; i++) {
            adjList[i] = new ArrayList<>();
        }
    }

    void addEdge(int v1, int v2) {
        adjList[v1].add(v2);
        adjList[v2].add(v1);
    }

    // BFS from source vertex
    void bfs(int source) {
        boolean[] visited = new boolean[numVertices];
        Queue<Integer> queue = new LinkedList<>();

        visited[source] = true;
        queue.offer(source);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print(vertex + " ");
        }
    }
}
```

```

        for (int neighbor : adjList[vertex]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}

// BFS to find shortest path
int[] bfsShortestPath(int source, int dest) {
    boolean[] visited = new boolean[numVertices];
    int[] parent = new int[numVertices];
    Queue<Integer> queue = new LinkedList<>();

    visited[source] = true;
    queue.offer(source);

    while (!queue.isEmpty()) {
        int vertex = queue.poll();
        if (vertex == dest) {
            // Reconstruct path
            return reconstructPath(parent, source, dest);
        }

        for (int neighbor : adjList[vertex]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                parent[neighbor] = vertex;
                queue.offer(neighbor);
            }
        }
    }
    return null; // No path exists
}

int[] reconstructPath(int[] parent, int source, int dest) {
    List<Integer> path = new ArrayList<>();
    for (int at = dest; at != source; at = parent[at]) {
        path.add(at);
    }
    path.add(source);
    Collections.reverse(path);
    return path.stream().mapToInt(i -> i).toArray();
}
}

```

BFS Time and Space Complexity

Aspect	Complexity
Time	$O(V + E)$
Space	$O(V)$

BFS Applications

Application	Use Case
Shortest path	Unweighted graph shortest path
Level order traversal	Tree level-by-level
Connected components	Find all reachable nodes
Bipartite checking	Can graph be 2-colored?
Cycle detection	For undirected graphs

4.4 Depth-First Search (DFS)

Definition

DFS explores as far as possible along each branch before backtracking. Uses a **stack** (or recursion).

DFS on Graph:

```
  0
 /|\
1 2 3
| | |
4 5 6
```

DFS Order (starting from 0):

0, 1, 4, backtrack to 0, 2, 5, backtrack to 0, 3, 6

Result: 0, 1, 4, 2, 5, 3, 6 (one possible order)

DFS Implementation (Recursive)

```
class GraphDFS {
    private List<Integer>[] adjList;
    private int numVertices;
    private boolean[] visited;

    GraphDFS(int numVertices) {
        this.numVertices = numVertices;
        adjList = new ArrayList[numVertices];
        for (int i = 0; i < numVertices; i++) {
            adjList[i] = new ArrayList<>();
        }
    }
}
```

```

    }
    visited = new boolean[numVertices];
}

void addEdge(int v1, int v2) {
    adjList[v1].add(v2);
    adjList[v2].add(v1);
}

void dfs(int start) {
    dfsRecursive(start);
}

private void dfsRecursive(int vertex) {
    visited[vertex] = true;
    System.out.print(vertex + " ");

    for (int neighbor : adjList[vertex]) {
        if (!visited[neighbor]) {
            dfsRecursive(neighbor);
        }
    }
}

// Reset visited array
void resetVisited() {
    Arrays.fill(visited, false);
}

// DFS with timestamp (discovery/finish time)
int time = 0;
int[] discoveryTime;
int[] finishTime;

void dfsWithTime(int start) {
    discoveryTime = new int[numVertices];
    finishTime = new int[numVertices];
    dfsTimeRecursive(start);
}

private void dfsTimeRecursive(int vertex) {
    visited[vertex] = true;
    time++;
    discoveryTime[vertex] = time;

    for (int neighbor : adjList[vertex]) {
        if (!visited[neighbor]) {

```

```

        dfsTimeRecursive(neighbor);
    }
}

time++;
finishTime[vertex] = time;
}
}

```

DFS Implementation (Iterative)

```

void dfsIterative(int start) {
    boolean[] visited = new boolean[numVertices];
    Stack<Integer> stack = new Stack<>();

    stack.push(start);

    while (!stack.isEmpty()) {
        int vertex = stack.pop();

        if (!visited[vertex]) {
            visited[vertex] = true;
            System.out.print(vertex + " ");

            // Add neighbors in reverse order to maintain order
            List<Integer> neighbors = adjList[vertex];
            for (int i = neighbors.size() - 1; i >= 0; i--) {
                if (!visited[neighbors.get(i)]) {
                    stack.push(neighbors.get(i));
                }
            }
        }
    }
}

```

DFS Time and Space Complexity

Aspect	Complexity
Time	$O(V + E)$
Space	$O(V)$ (visited array + stack)

DFS Applications

Application	Use Case
Topological sort	Ordering DAG vertices
Cycle detection	For directed graphs
Path finding	Find any path between two nodes
Connected components	Find strongly connected components
Maze solving	Find path from start to exit

4.5 BFS vs DFS Comparison

Aspect	BFS	DFS
Data structure	Queue	Stack (or recursion)
Exploration	Layer by layer	Deep along each branch
Shortest path	Yes (unweighted)	No
Memory	$O(V) + O(V)$ for queue	$O(V)$ for stack
Time	$O(V + E)$	$O(V + E)$
Completeness	Finds solution if exists	Finds solution if exists

When to Use BFS vs DFS

Use Case	Choose
Shortest path (unweighted)	BFS
All paths enumeration	DFS
Topological sort	DFS
Cycle detection	DFS
Level order traversal	BFS
Maze solving	Either

4.6 Minimum Spanning Tree (MST)

Definition

A **spanning tree** connects all vertices with minimum number of edges ($V-1$ edges) and no cycles.
A **Minimum Spanning Tree** is the spanning tree with minimum total weight.

GRAPH:	MST:
A--2--B	A--2--B
1 3	
C D	1 3
4 5	C---D
E--6--F	(MST has no cycles)

All vertices connected with minimum total weight = $2+1+3+4 = 10$

Properties of MST

Property	Description
Unique	If all edge weights are distinct
V-1 edges	Always has exactly V-1 edges
No cycles	Acyclic by definition
Minimum total	Sum of edge weights is minimum

Prim's Algorithm

Greedy algorithm that grows MST by adding cheapest edge from tree to non-tree vertex.

```
class PrimMST {
    void primMST(int[] [] graph, int V) {
        boolean[] inMST = new boolean[V];
        int[] key = new int[V]; // Minimum weight to connect to MST
        Arrays.fill(key, Integer.MAX_VALUE);
        int[] parent = new int[V];

        key[0] = 0;
        parent[0] = -1;

        for (int count = 0; count < V - 1; count++) {
            int u = minKey(key, inMST, V);
            inMST[u] = true;

            for (int v = 0; v < V; v++) {
                if (graph[u][v] != 0 && !inMST[v] && graph[u][v] < key[v]) {
                    key[v] = graph[u][v];
                    parent[v] = u;
                }
            }
        }

        printMST(parent, graph, V);
    }

    int minKey(int[] key, boolean[] inMST, int V) {
        int min = Integer.MAX_VALUE, minIndex = -1;
        for (int v = 0; v < V; v++) {
            if (!inMST[v] && key[v] < min) {
                min = key[v];
                minIndex = v;
            }
        }
        return minIndex;
    }
}
```

```

    }

    void printMST(int[] parent, int[][] graph, int V) {
        System.out.println("Edge \tWeight");
        int totalWeight = 0;
        for (int i = 1; i < V; i++) {
            System.out.println(parent[i] + " - " + i + "\t" + graph[i][parent[i]]);
            totalWeight += graph[i][parent[i]];
        }
        System.out.println("Total weight: " + totalWeight);
    }
}

```

Prim's Algorithm Visualization

Step 1: Start with vertex A (key=0, others=INF)
 A(0) B(infinity) C(infinity) D(infinity)
 [A] is in MST

Step 2: Update neighbors of A
 A(0) B(2) C(1) D(infinity)
 Add C (smallest key)

Step 3: Update neighbors of C
 A(0) B(2) C(1) D(3)
 Add B (key=2)

Step 4: Add D (key=3)

MST: A-C, C-B, B-D = $0+1+3 = 4$

Kruskal's Algorithm

Greedy algorithm that sorts all edges by weight and adds smallest edge that doesn't create cycle.

```

class KruskalMST {
    class Edge implements Comparable<Edge> {
        int src, dest, weight;
        public int compareTo(Edge edge) {
            return this.weight - edge.weight;
        }
    }

    class UnionFind {
        int[] parent, rank;
        UnionFind(int n) {
            parent = new int[n];
        }
    }
}

```

```

        rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }

    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }

    boolean union(int x, int y) {
        int px = find(x), py = find(y);
        if (px == py) return false; // Same set, would create cycle

        if (rank[px] < rank[py]) parent[px] = py;
        else if (rank[px] > rank[py]) parent[py] = px;
        else { parent[py] = px; rank[px]++; }
        return true;
    }
}

void kruskalMST(Edge[] edges, int V) {
    Arrays.sort(edges);

    UnionFind uf = new UnionFind(V);
    int totalWeight = 0;

    System.out.println("MST Edges:");
    for (Edge edge : edges) {
        if (uf.union(edge.src, edge.dest)) {
            System.out.println(edge.src + " - " + edge.dest + "\t" + edge.weight);
            totalWeight += edge.weight;
        }
    }
    System.out.println("Total weight: " + totalWeight);
}
}

```

Kruskal's Algorithm Visualization

Edges sorted by weight: (0,1)=1, (0,2)=2, (1,2)=3, (1,3)=4, (2,3)=5

Pick (0,1)=1:	0 -- 1	Union sets {0,1} and {2,3}
Pick (0,2)=2:	0 -- 1	Union sets {0,1,2} and {3}
	2	
Pick (1,3)=4:	0 -- 1	Union sets {0,1,2,3} = done (V-1=3 edges)

MST: $(0,1) + (0,2) + (1,3) = 1 + 2 + 4 = 7$

Prim vs Kruskal

Aspect	Prim	Kruskal
Approach	Grow from one vertex	Grow from edges
Greedy choice	Cheapest from tree	Cheapest globally
Data structure	Priority queue	Sort + Union-Find
Time (dense)	$O(V^2)$ better	$O(E \log E)$ worse
Time (sparse)	$O(E \log V)$	$O(E \log E)$
Best for	Dense graphs	Sparse graphs

4.7 Dijkstra's Shortest Path Algorithm

Definition

Dijkstra's algorithm finds shortest paths from a source vertex to all other vertices in a weighted graph with non-negative weights.

Example:

```
A --5-- B
|       |
2       3
|       |
C --4-- D
```

Shortest path from A to D:

A→C = 2

C→D = 4, total = 6

A→B→D = 5 + 3 = 8

So A→C→D is shorter ($6 < 8$)

Dijkstra Implementation

```
class Dijkstra {
    void dijkstra(int[][] graph, int src, int V) {
        int[] dist = new int[V];
        boolean[] visited = new boolean[V];

        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        for (int count = 0; count < V - 1; count++) {
            int u = minDistance(dist, visited, V);
```

```

        visited[u] = true;

        for (int v = 0; v < V; v++) {
            if (!visited[v] && graph[u][v] != 0 &&
                dist[u] != Integer.MAX_VALUE &&
                dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printSolution(dist, V);
}

int minDistance(int[] dist, boolean[] visited, int V) {
    int min = Integer.MAX_VALUE, minIndex = -1;
    for (int v = 0; v < V; v++) {
        if (!visited[v] && dist[v] < min) {
            min = dist[v];
            minIndex = v;
        }
    }
    return minIndex;
}

void printSolution(int[] dist, int V) {
    System.out.println("Vertex \t Distance from Source");
    for (int i = 0; i < V; i++) {
        System.out.println(i + " \t\t " + dist[i]);
    }
}
}

```

Dijkstra with Priority Queue (Optimized)

```

class DijkstraPQ {
    void dijkstra(List<Edge>[] graph, int src, int V) {
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        PriorityQueue<Pair> pq = new PriorityQueue<>(V, (a, b) -> a.dist - b.dist);
        pq.add(new Pair(0, src));

        while (!pq.isEmpty()) {
            Pair p = pq.poll();

```

```

        if (p.dist > dist[p.vertex]) continue;

        for (Edge e : graph[p.vertex]) {
            if (dist[p.vertex] + e.weight < dist[e.dest]) {
                dist[e.dest] = dist[p.vertex] + e.weight;
                pq.add(new Pair(dist[e.dest], e.dest));
            }
        }
    }

    printSolution(dist, V);
}
}

```

Dijkstra's Algorithm Visualization

Graph: Starting from A

```

A --10-- B
|       |
2       3
|       |
C --4-- D

```

Step 1: dist[A]=0, others=INF
A(0), B(10), C(2), D(INF)

Step 2: Pick C (smallest unvisited)
Update B: dist[A]+2 < dist[B]? No (10 stays)
A(0), B(10), C(2), D(6) because 2+4=6

Step 3: Pick A (but already visited with smallest) or D
Actually next is D (dist=6)
D neighbors: B via D? 6+3=9 < 10, so B=9

Step 4: Pick B (dist=9)

Final: A=0, C=2, D=6, B=9

Dijkstra Limitations

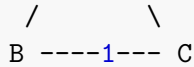
// Dijkstra DOES NOT work with negative edge weights!

// Counter-example:

```

      A
     / \
    1  -2

```



```

// Dijkstra from A:
// Step 1: A=0, B=1, C=-2 (wrong!)
// Correct: A=0, B=1, C=-1 (via B)

```

[WARNING] Professor Trap: Dijkstra requires non-negative edge weights!
For negative weights, use Bellman-Ford algorithm instead.

Dijkstra Time Complexity

Implementation	Time	Space
Simple (array)	$O(V^2)$	$O(V)$
With priority queue	$O((V+E) \log V)$	$O(V)$

4.7 Bellman-Ford Algorithm

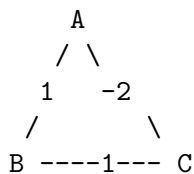
Definition

The **Bellman-Ford algorithm** finds shortest paths from a source vertex to all other vertices, even when edges can have negative weights. It also detects negative weight cycles.

KEY DIFFERENCE FROM DIJKSTRA:

- Dijkstra: Cannot handle negative weights (gives wrong answer)
- Bellman-Ford: Handles negative weights correctly, detects negative cycles

Example where Dijkstra fails:



Dijkstra from A:

- A=0, B=1, C=-2 (wrong, because B→C=-2 exists)

Bellman-Ford from A:

- Correctly computes A=0, B=1, C=-1 (via B: 1 + (-2))

Why Bellman-Ford Works

```

// Key insight: After V-1 iterations, shortest paths are found
// Because any simple path has at most V-1 edges

```

```

// For path A -> B -> C -> D:
// A -> B is 1 edge
// A -> B -> C is 2 edges
// A -> B -> C -> D is 3 edges
// Max edges in shortest simple path = V - 1

// So after V-1 relaxations, all shortest paths are finalized

```

Bellman-Ford Implementation

```

class BellmanFord {
    // Edge structure
    static class Edge {
        int src, dest, weight;
        Edge(int src, int dest, int weight) {
            this.src = src;
            this.dest = dest;
            this.weight = weight;
        }
    }

    static void bellmanFord(int[][] graph, int V, int E, int src) {
        Edge[] edges = new Edge[E];
        int edgeCount = 0;

        // Convert adjacency matrix to edge list
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (graph[i][j] != 0 && graph[i][j] != Integer.MAX_VALUE) {
                    edges[edgeCount++] = new Edge(i, j, graph[i][j]);
                }
            }
        }

        // Initialize distances
        int[] dist = new int[V];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        // Step 1: Relax all edges V-1 times
        for (int i = 0; i < V - 1; i++) {
            for (int j = 0; j < E; j++) {
                int u = edges[j].src;
                int v = edges[j].dest;
                int weight = edges[j].weight;

                if (dist[u] != Integer.MAX_VALUE && dist[u] + weight < dist[v]) {

```

```

        dist[v] = dist[u] + weight;
    }
}

// Step 2: Check for negative weight cycles
boolean hasNegativeCycle = false;
for (int j = 0; j < E; j++) {
    int u = edges[j].src;
    int v = edges[j].dest;
    int weight = edges[j].weight;

    if (dist[u] != Integer.MAX_VALUE && dist[u] + weight < dist[v]) {
        hasNegativeCycle = true;
        break;
    }
}

// Print result
System.out.println("Vertex \t Distance from Source");
for (int i = 0; i < V; i++) {
    if (dist[i] == Integer.MAX_VALUE) {
        System.out.println(i + " \t\t INF");
    } else {
        System.out.println(i + " \t\t " + dist[i]);
    }
}

if (hasNegativeCycle) {
    System.out.println("Graph contains negative weight cycle!");
}
}
}

```

Bellman-Ford Step-by-Step Example

GRAPH:

```

A --2--> B
|         |
1         3
v         v
C --(-2)--> D

```

Edges: A->B(2), A->C(1), B->D(3), C->D(-2)

Initialize: dist[A]=0, dist[B]=INF, dist[C]=INF, dist[D]=INF

Iteration 1 (relax all edges):

- A->B: $\text{dist}[B] = 0 + 2 = 2$
- A->C: $\text{dist}[C] = 0 + 1 = 1$
- B->D: $\text{dist}[B]=\text{INF}$, skip
- C->D: $\text{dist}[D] = 1 + (-2) = -1$

dist: A=0, B=2, C=1, D=-1

Iteration 2:

- B->D: $\text{dist}[D] = 2 + 3 = 5 > -1$, no update

Iteration 3 ($V-1 = 3$, so one more iteration):

- No updates (already converged)

Final: A=0, B=2, C=1, D=-1

Verification:

- Shortest path A->D = -1 (via A->C->D = $1 + (-2)$)
- Shortest path A->B = 2 (direct)
- Shortest path A->C = 1 (direct)

Negative Weight Cycle Detection

GRAPH with NEGATIVE CYCLE:

```
A --1--> B
^         |
|         1
|         v
D --(-3)-- C
```

Path A->B->C->D->A = $1 + 1 + (-3) + 1 = 0$, not negative...

Let's try:

```
A --1--> B
^         |
|        -2
|         v
D <---3-- C
```

Path A->B->C->D = $1 + (-2) + 3 = 2$

Actually negative cycle:

```
A --1--> B
^         |
|         2
|         v
D <-(-4)-- C
```

$A \rightarrow B \rightarrow C \rightarrow D = 1 + 2 + (-4) = -1$ (decreases!)

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A = 1 + 2 + (-4) + 1 = 0$

Bellman-Ford vs Dijkstra

Aspect	Bellman-Ford	Dijkstra
Negative weights	Handles correctly	Does NOT work
Time complexity	$O(V * E)$	$O((V+E) \log V)$
Space complexity	$O(V)$	$O(V)$
Negative cycle detection	Yes	No
Speed	Slower for dense graphs	Faster

When to Use Bellman-Ford

Scenario	Algorithm
All edge weights positive	Dijkstra (faster)
Some negative edge weights	Bellman-Ford
Need negative cycle detection	Bellman-Ford
Graph is dense ($E \sim V^2$)	Bellman-Ford may be simpler
Graph is sparse	Dijkstra with PQ preferred

4.8 Topological Sort

Definition

Topological sort orders vertices in a DAG so that for every directed edge (u, v) , u comes before v .

DAG (Course Prerequisites):

```
CS101 (no prereq)
  |
  v
CS201 (needs CS101)
  |
  v
CS301 (needs CS201)
```

Valid topological orderings:

CS101, CS201, CS301

CS101, CS201, CS301

NOT valid:

CS201, CS101, CS301 (CS201 before CS101 - violates prerequisite)

Topological Sort using DFS

```
class TopologicalSort {
    List<Integer>[] adjList;
    int numVertices;

    TopologicalSort(int numVertices) {
        this.numVertices = numVertices;
        adjList = new ArrayList[numVertices];
        for (int i = 0; i < numVertices; i++) {
            adjList[i] = new ArrayList<>();
        }
    }

    void topologicalSortDFS() {
        boolean[] visited = new boolean[numVertices];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < numVertices; i++) {
            if (!visited[i]) {
                topologicalSortDFS(i, visited, stack);
            }
        }

        System.out.print("Topological Order: ");
        while (!stack.isEmpty()) {
            System.out.print(stack.pop() + " ");
        }
    }

    private void topologicalSortDFS(int vertex, boolean[] visited, Stack<Integer> stack) {
        visited[vertex] = true;

        for (int neighbor : adjList[vertex]) {
            if (!visited[neighbor]) {
                topologicalSortDFS(neighbor, visited, stack);
            }
        }

        stack.push(vertex); // Push after exploring all neighbors
    }
}
```

Topological Sort using Kahn's Algorithm (BFS)

```
class TopologicalSortKahn {
    void topologicalSortBFS(int V, List<Integer>[] adjList) {
```

```

int[] inDegree = new int[V];

// Calculate in-degrees
for (int i = 0; i < V; i++) {
    for (int neighbor : adjList[i]) {
        inDegree[neighbor]++;
    }
}

Queue<Integer> queue = new LinkedList<>();
for (int i = 0; i < V; i++) {
    if (inDegree[i] == 0) {
        queue.offer(i); // Vertices with no prerequisites
    }
}

int count = 0;
List<Integer> result = new ArrayList<>();

while (!queue.isEmpty()) {
    int vertex = queue.poll();
    result.add(vertex);

    for (int neighbor : adjList[vertex]) {
        inDegree[neighbor]--;
        if (inDegree[neighbor] == 0) {
            queue.offer(neighbor);
        }
    }
    count++;
}

if (count != V) {
    System.out.println("Graph has cycle! Topological sort not possible.");
    return;
}

System.out.println("Topological Order: " + result);
}
}

```

Topological Sort Properties

Property	Value
Exists if and only if	Graph is DAG (no cycles)
Not unique	Multiple valid orderings possible

Property	Value
Time complexity	$O(V + E)$
Space complexity	$O(V)$

Topological Sort Applications

Application	Example
Course scheduling	Prerequisites before courses
Build systems	Compile order for files
Task scheduling	Order dependent tasks
Dependency resolution	Package installation order

4.9 Graph Cycle Detection

Detect Cycle in Undirected Graph (DFS)

```

boolean hasCycleUndirected(List<Integer>[] graph, int V) {
    boolean[] visited = new boolean[V];

    for (int i = 0; i < V; i++) {
        if (!visited[i]) {
            if (hasCycleDFS(i, visited, -1)) {
                return true;
            }
        }
    }
    return false;
}

boolean hasCycleDFS(int vertex, boolean[] visited, int parent) {
    visited[vertex] = true;

    for (int neighbor : graph[vertex]) {
        if (!visited[neighbor]) {
            if (hasCycleDFS(neighbor, visited, vertex)) {
                return true;
            }
        } else if (neighbor != parent) {
            // visited neighbor that's not parent -> cycle!
            return true;
        }
    }
    return false;
}

```

Detect Cycle in Directed Graph (DFS with 3 states)

```
enum State { UNVISITED, IN_PROGRESS, VISITED }

boolean hasCycleDirected(List<Integer>[] graph, int V) {
    State[] states = new State[V];
    Arrays.fill(states, State.UNVISITED);

    for (int i = 0; i < V; i++) {
        if (states[i] == State.UNVISITED) {
            if (hasCycleDFS(i, graph, states)) {
                return true;
            }
        }
    }
    return false;
}

boolean hasCycleDFS(int vertex, List<Integer>[] graph, State[] states) {
    states[vertex] = State.IN_PROGRESS;

    for (int neighbor : graph[vertex]) {
        if (states[neighbor] == State.IN_PROGRESS) {
            return true; // Back edge found -> cycle!
        }
        if (states[neighbor] == State.UNVISITED) {
            if (hasCycleDFS(neighbor, graph, states)) {
                return true;
            }
        }
    }

    states[vertex] = State.VISITED;
    return false;
}
```

4.10 Edge Cases & Traps

Trap 1: Directed vs Undirected Confusion

```
// Adding edge for undirected graph
addEdge(v1, v2) {
    adj[v1].add(v2);
    adj[v2].add(v1); // Must add both ways!
}

// Adding edge for directed graph
```

```
addEdge(v1, v2) {
    adj[v1].add(v2); // Only one direction!
}
```

[WARNING] **Professor Trap:** Undirected graphs need edges in BOTH directions!

Trap 2: BFS for Shortest Path Only in Unweighted Graphs

```
// BFS gives shortest path for UNWEIGHTED graphs
// For WEIGHTED graphs, use Dijkstra!

// WRONG: BFS for weighted graph shortest path
graph = weighted graph with distances
bfs(source); // May give wrong path!

// CORRECT: Dijkstra for weighted
dijkstra(source);
```

[WARNING] **Professor Trap:** BFS gives shortest path only when all edges have equal weight!

Trap 3: Dijkstra with Negative Weights

```
// Dijkstra produces WRONG results with negative weights
// Example where Dijkstra fails:

// A -> B with weight 2
// A -> C with weight 3
// B -> C with weight -2

// Correct shortest path A->C = 0 (via B: 2 + (-2))
// Dijkstra would compute A->C = 3 (direct)
```

[WARNING] **Professor Trap:** Use Bellman-Ford for graphs with negative weights!

Trap 4: Graph with Multiple Connected Components

```
// BFS/DFS from one vertex won't visit all vertices!
graph with 2 disconnected components
bfs(0); // Only visits component containing 0!

// Must check all vertices:
for (int i = 0; i < V; i++) {
    if (!visited[i]) {
        bfs(i); // Start new BFS from unvisited vertex
    }
}
```

[WARNING] **Professor Trap:** Always iterate through all vertices to handle disconnected graphs!

Trap 5: Stack Overflow in DFS for Large Graphs

```
// Recursive DFS can overflow for deep graphs
dfs(vertex) {
    dfs(vertex.left);
    dfs(vertex.right);
}
// Graph with 100,000 vertices deep -> StackOverflowError!

// Fix: Use iterative DFS with explicit stack
```

[WARNING] **Professor Trap:** For very deep graphs, use iterative DFS instead of recursive!

4.11 Practice Questions

MCQ 1: BFS Data Structure

What data structure does BFS use? - A) Stack - B) Queue - C) Priority Queue - D) Deque

MCQ 2: Dijkstra's Algorithm

Dijkstra's algorithm works for graphs with: - A) Any weights - B) Only positive weights - C) Only negative weights - D) No weights

MCQ 3: MST Edges

A minimum spanning tree with V vertices has how many edges? - A) V - B) V - 1 - C) 2V - D) V²

MCQ 4: Topological Sort

Topological sort can only be performed on: - A) Any graph - B) Directed acyclic graph (DAG) - C) Undirected graph - D) Complete graph

Short Answer 1

Explain the difference between BFS and DFS. When would you choose one over the other?

Short Answer 2

Why does Prim's algorithm guarantee a minimum spanning tree?

Board Coding 1

Implement BFS to find shortest path in an unweighted graph.

Board Coding 2

Write code to detect if a directed graph has a cycle.

Board Coding 3

Implement topological sort using Kahn's algorithm (BFS-based).

Oral Explanation

Explain how Dijkstra's algorithm works step by step on a small weighted graph.

Solutions

MCQ 1: B) Queue - BFS explores level by level using queue

MCQ 2: B) Only positive weights - Negative weights can cause incorrect results

MCQ 3: B) $V - 1$ - MST connects all V vertices with no cycles, which requires $V-1$ edges

MCQ 4: B) DAG - Topological sort requires no cycles (DAG)

Short Answer 1: BFS uses queue, explores layer by layer, finds shortest path in unweighted graphs. DFS uses stack/recursion, explores deep before backtracking, good for path finding and topological sort. Choose BFS for shortest path, DFS for exploring all paths or when stack is natural fit.

Short Answer 2: Prim's algorithm is greedy - at each step it adds the minimum weight edge that connects a vertex in the tree to a vertex outside. This maintains the cut property: for any cut, the minimum weight crossing edge is in some MST. By always choosing such edges, the result is optimal.

Cheat-Sheet

Graph Representations

Representation	Space	Edge Check	Neighbor List
Adjacency Matrix	$O(V^2)$	$O(1)$	$O(V)$
Adjacency List	$O(V + E)$	$O(\text{degree})$	$O(\text{degree})$

Traversals

Algorithm	Data Structure	Use Case
BFS	Queue	Shortest path (unweighted), level order
DFS	Stack/Recursion	Path finding, topological sort, cycle detection

Algorithms

Algorithm	Purpose	Time
BFS	Shortest path (unweighted)	$O(V + E)$
DFS	Exploration	$O(V + E)$
Prim	MST	$O(V^2)$ or $O(E \log V)$
Kruskal	MST	$O(E \log E)$
Dijkstra	Shortest path	$O(V^2)$ or $O(E \log V)$

Algorithm	Purpose	Time
Topological Sort	DAG ordering	$O(V + E)$

Review Checklist

- ☐ **Graph types:** Can you distinguish directed/undirected, weighted/unweighted?
- ☐ **Representations:** Do you know when to use matrix vs list?
- ☐ **BFS:** Can you implement BFS and explain level-by-level exploration?
- ☐ **DFS:** Can you implement DFS recursively and iteratively?
- ☐ **MST:** Do you understand Prim's and Kruskal's greedy approaches?
- ☐ **Dijkstra:** Can you trace through the algorithm step by step?
- ☐ **Topological sort:** Can you order DAG vertices correctly?
- ☐ **Traps:** Are you aware of negative weights, disconnected graphs, and stack overflow issues?
- ☐ **When to use:** Can you choose the right algorithm for given problem?